14 haftalık yol haritasında 7. hafta

ALGORİTMA VE PROGRAMLAMA – I · DERS NOTU

HAFTA 07

Döngüler II: İç İçe Döngüler, Problem Çözme Desenleri ve Ara Proje

Izgara modeli ve yineleme sayısı; satır–sütun desenleri; asallık ve √n; seriler ve ε; clock() ile ölçüm; menü döngüsü ve ATM ara projesi

MODÜL III · KONTROL AKIŞI · H5–H7

HAFTANIN BÜYÜK SORUSU

Bir sayının asal olup olmadığını en hızlı nasıl anlarız ve "daha hızlı" iddiamızı nasıl ölçeriz?

Dr. Öğr. Üyesi Mehmet Sevri
Recep Tayyip Erdoğan Üniversitesi · Mühendislik ve Mimarlık Fakültesi · Bilgisayar Mühendisliği Bölümü
Bilgisayar Mühendisliği 1. sınıf · 2026–2027 Güz yarıyılı · Sürüm 1.0 · Eylül 2026
`Derleme ortamı: GCC (C17) · gcc -std=c17 -Wall -Wextra -pedantic`

İçindekiler

1. Haftanın Büyük Sorusu ve Öğrenme Çıktıları ... 3
1.1 Öğrenme çıktıları ... 3
2. Geçen Haftadan: Tek Döngüden İç İçe Döngüye ... 3
3. Kavramsal Anlatım ... 4
3.1 İç içe döngü: ızgara modeli ... 4
3.2 Kaç yineleme? n·m ve n(n+1)/2 ... 5
3.3 Satır–sütun düşüncesi: desenleri formülle çizmek ... 5
3.4 Problem çözme desenleri: H6'nın desenleri iç içe ... 6
3.5 Asallık: naif test, √n sınırı ve tek bölenler ... 7
3.6 ε ile biten döngüler: seriler ... 8
3.7 İç içe döngüden çıkış: bayrak, break ve goto ... 8
3.8 Menü döngüsü: do-while + switch ... 9
3.9 Kaputun altında: "daha hızlı" iddiasını ölçmek ... 10
4. Çözümlü Örnekler: Örnek Merdiveni ... 12
4.1 ★ basamaklar: dikdörtgen, üçgenler ve çarpım tablosu (Örnek 1–3) ... 12
4.2 Amiral örnek — Örnek 4 ★★: piramit (boşluk + yıldız) ... 12
4.3 Örnek 5 ★★ — Floyd üçgeni ve sayı üçgeni ... 14
4.4 Asallık: naif test ve 1–1000 arası asallar (Örnek 6–7) ... 15
4.5 Örnek 8 ★★ — Mükemmel sayılar ve Armstrong sayıları ... 15
4.6 Kayan değişkenler ve ardışık bölme (Örnek 9–10) ... 16
4.7 Amiral örnek — Örnek 11 ★★★: iyileştirilmiş asallık ve süre ölçümü ... 17
4.8 Seriler, kaba kuvvet ve arama uzayı (Örnek 12–13) ... 19
4.9 Amiral örnek — Örnek 14 ★★★: menü güdümlü ATM simülasyonu (ara proje) ... 20
5. Hata Avcısı ... 22
6. Mühendis Gözüyle: Doğru, Ölçülmüş, Sağlam ... 23
7. Mini-OBS v0.7 ve Ara Proje ... 24
7.1 Mini-OBS v0.7: menü güdümlü ana döngü ... 24
7.2 Ara proje (eşli): yönergenin özeti ... 25
8. Alıştırmalar ve Atölye ... 25
9. Kendini Değerlendirme ... 28
10. AP-2 Köprüsü ve Merak Kutusu ... 28
11. Özet, Kavram Haritası ve Terim Sözlüğü ... 29
12. Kaynaklar ... 30

1. Haftanın Büyük Sorusu ve Öğrenme Çıktıları

Bir sayının asal olup olmadığını en hızlı nasıl anlarız ve "daha hızlı" iddiamızı nasıl ölçeriz?

H6'da tek bir döngüyle toplamayı, saymayı, en büyüğü bulmayı öğrendik. Bu hafta döngüleri **iç içe** kuruyoruz: bir döngünün gövdesinde başka bir döngü. Böylece iki boyutlu çıktılar (tablolar, desenler), "her aday için her böleni dene" türünden aramalar ve menü güdümlü programlar yazabileceğiz. İç içe döngünün bedeli yineleme sayısının hızla büyümesidir: 0 ile 100 000 arasındaki asalları en saf yöntemle sayan programımız **455 189 150** bölme yaptı; aynı işi biraz düşünerek yapan sürüm **1 345 442** bölmeyle bitirdi (Bölüm 3.9). "Daha hızlı" iddiasını bu hafta sezgiyle değil **ölçerek** savunacağız.

> GERÇEK DÜNYADAN · **Mayıs 1997: Deep Blue ve kaba kuvvetin gücü**
>
> IBM'in satranç bilgisayarı Deep Blue, Mayıs 1997'de New York'taki Equitable Center'da dünya şampiyonu Garri Kasparov'la altı oyunluk bir rövanş maçı oynadı ve maçı 3½–2½ kazandı; böylece standart turnuva süre denetimlerinde yürürlükteki bir dünya şampiyonunu yenen ilk bilgisayar sistemi oldu (IBM, t.y.).
>
> Sistemi tasarlayanların anlatımıyla 1997 sürümü, 30 düğümlü bir IBM RS/6000 SP bilgisayarına bağlı **480 özel satranç yongasından** oluşuyordu; bir dakikadan uzun aramalarda ortalama hızı **saniyede 126 milyon konum**, maçta gözlenen en yüksek sürekli hızı saniyede 330 milyon konumdu (Campbell vd., 2002). Bu, kaba kuvvetin (brute force) gücüdür: olasılıkları tek tek değerlendirmek, yeterince hızlıysanız insan sezgisine kafa tutabilir.
>
> Ama Deep Blue saf bir "her şeyi dene" programı değildi. Değerlendirme fonksiyonu donanımda gerçeklenmişti ve sekiz binin üzerinde özellik içeriyordu; arama, önemli görünen hamle dizilerini daha derin inceleyen **seçici arama uzatmalarına** dayanıyordu (Campbell vd., 2002). Çünkü kaba kuvvetin bir sınırı vardır: Shannon'ın (1950) kaba kestirimiyle bir hamle çiftinde yaklaşık 1000 olasılık varsa, d hamle çifti ileriye bakmak 1000^d konum demektir. Saniyede 126 milyon konumla bile 6 hamle çifti derinliğe tek tek bakmak yaklaşık **250 yıl** sürerdi (Bölüm 4.8; grafiği sunumda ve web'de).
>
> Bu haftanın küçük ölçekli deneyleri aynı dersi verir: arama uzayı iç içe döngülerle **çarpılarak** büyür; kazanç, bakılması gerekmeyen yerleri akıllıca elemekten gelir (√n sınırı, Pisagor'da daraltılmış sınırlar).

1.1 Öğrenme çıktıları

ÖÇ 7.1 İç içe döngülerin çalışma sırasını izler ve toplam yineleme sayısını hesaplar. *(Bölüm 3.1–3.2, Örnek 1–3, 5)*

ÖÇ 7.2 Satır–sütun düşüncesiyle iki boyutlu çıktı desenleri üretir. *(Bölüm 3.3, Örnek 4)*

ÖÇ 7.3 Sayı kuramı problemlerini (asallık, mükemmel sayı, Armstrong sayısı) döngülerle çözer; çözümü verimlilik açısından iyileştirir (√n sınırı). *(Bölüm 3.4–3.6, Örnek 6–13)*

ÖÇ 7.4 clock() ile çalışma süresini ölçerek iki çözümü deneysel olarak karşılaştırır. *(Bölüm 3.9, Örnek 11)*

ÖÇ 7.5 Menü güdümlü, kullanıcıyla etkileşimli bir program tasarlar ve gerçekler. *(Bölüm 3.8, Örnek 14, Bölüm 7)*

2. Geçen Haftadan: Tek Döngüden İç İçe Döngüye

H6'da üç döngüyü (`while`, `do-while`, `for`), sayaç ve nöbetçi değer kontrolünü, güvenli girdi döngüsünü ve altı döngü desenini (toplayıcı, sayaç, çarpım biriktirici, bayrak, en büyük/en küçük, basamak ayrıştırma; `H06_K06_desenler.c`) öğrendik. Bu hafta o desenleri **tekrar etmeyeceğiz, kullanacağız**: asallıkta bayrak, mükemmel sayıda toplayıcı, Armstrong'da basamak ayrıştırma ve çarpım biriktirici, menüde güvenli girdi döngüsü (`H06_12_gecerli_girdi.c`). Üç geri çağırma sorusu (programlar `kodlar/quiz/`):

TAHMİN ET · **Geri çağırma: üç soru**

- **S1 (üç döngü).** `H07_Q01_dongu_turleri.c`: `a = 5` ile `while (a < 3)`, `b = 5` ile `do { … } while (b < 3)` ve `for (int c = 0; c < 3; c += 2)`. Hangi harfler kaçar kez yazılır, `a` ve `b` son olarak kaçtır?
- **S2 (nöbetçi değer).** `H07_Q02_nobetci_toplam.c` okumayı döngü koşulunda yapan bir toplayıcı-sayaç döngüsüdür. Girdi `4 7 -1 9` ise çıktı nedir? `9` neden toplanmaz?
- **S3 (scanf dönüş değeri).** `H07_Q03_scanf_donus.c`'ye `abc` yazılırsa `r` ve `n` kaç olur? Ctrl+D (dosya sonu) basılırsa?

Yanıtlar (gerçek çıktılar). S1: `DFF 5 6` — `while` gövdesi koşul baştan yanlış olduğu için hiç çalışmaz; `do-while` gövdesi koşula bakılmadan bir kez çalışır (`b` 6 olur); `for` 0 ve 2 için iki tur döner. S2: `adet = 2, toplam = 11` — nöbetçi (-1) okunur okunmaz döngü biter, sonrasındaki `9` hiç okunmaz. S3: `abc` için "r = 0, n = 7", dosya sonunda "Bir sayı: r = -1, n = 7" (`EOF` = -1); başarısız okuma `n`'yi değiştirmez. Bu hafta ATM'de ve Mini-OBS'te bu üç bilgiyi aynı anda kullanacağız.

3. Kavramsal Anlatım

3.1 İç içe döngü: ızgara modeli

İç içe döngü (nested loop), gövdesinde başka bir döngü bulunan döngüdür. Kural tektir ve H6'dan bildiğimiz kuraldır: döngü gövdesi baştan sona çalışır. Gövdenin içinde bir döngü varsa, dış döngünün **her** turunda o iç döngü baştan başlar ve **sonuna kadar** çalışır; ancak ondan sonra dış döngü bir sonraki tura geçer.

Bir duvar saati iyi bir benzetmedir: akrep (dış döngü) bir saat ilerlerken yelkovan (iç döngü) 60 dakikanın tamamını dolaşır ve yeniden 0'dan başlar. Bir kilometre sayacında da sağdaki hane 0–9'u dolaşıp başa döndüğünde soldaki hane bir artar. İç değişken **hızlı**, dış değişken **yavaş** değişir.

H07_K01_izgara_sayac.c · satır 15–23

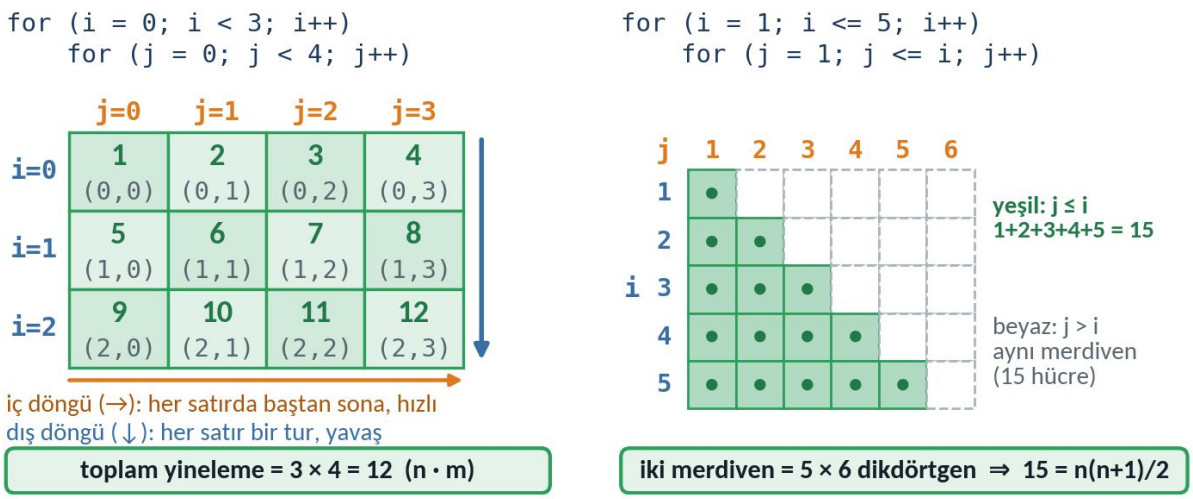
```
15      int adim = 0;
16
17      for (int i = 0; i < SATIR; i++) {      /* dış: yavaş değişir */
18          for (int j = 0; j < SUTUN; j++) {  /* iç: hızlı değişir */
19              adim++;
20              printf("(%d,%d) ", i, j);
21          }
22          printf("<- i = %d satırı bitti\n", i);
23      }
```

ÇIKTI

```
$ ./izgara_sayac
(0,0) (0,1) (0,2) (0,3) <- i = 0 satırı bitti
(1,0) (1,1) (1,2) (1,3) <- i = 1 satırı bitti
(2,0) (2,1) (2,2) (2,3) <- i = 2 satırı bitti
İç gövde 12 kez çalıştı = 3 x 4
```

Bu çalışmayı bir **ızgara** olarak düşünmek iç içe döngünün en güçlü zihinsel modelidir (Şekil 7.1): `i` satırı, `j` sütunu seçer; **her hücre, iç gövdenin bir yinelemesidir**. Hücreler satır satır, soldan sağa doldurulur. Dikdörtgen bir ızgarada toplam yineleme **satır × sütun** kadardır. İç sınır dış değişkene bağlanırsa (`j <= i`) ızgara **üçgene** dönüşür.

Şekil 7.1
Izgara modeli: solda 3 × 4 dikdörtgen (hücredeki sayı yineleme sırası), sağda j ≤ i üçgeni ve onu 5 × 6 dikdörtgene tamamlayan ikinci merdiven



Koşul denetimlerini de sayabiliriz (H6'daki "n tur için n + 1 denetim" kuralı her döngüye ayrı uygulanır): dış koşul 3 + 1 = 4 kez sınanır; iç döngü 3 kez baştan başlar ve her başlayışında 4 + 1 = 5 kez sınanır, toplam 15. Webdeki "Yineleme sayacı" simülasyonu bu sayıları her adımda gösterir.

3.2 Kaç yineleme? n·m ve n(n+1)/2

İç içe döngünün maliyetini önceden kestirmek için iç gövdenin kaç kez çalışacağını sayarız. **Dikdörtgen** (iç sınır dış değişkenden bağımsız): dış döngü n, iç döngü m tur dönüyorsa toplam **n · m**. **Üçgen** (iç sınır dış değişkene bağlı): `for (i = 1; i <= n; i++) for (j = 1; j <= i; j++)` için toplam 1 + 2 + … + n = **n(n + 1)/2**. Şekil 7.1'in sağ yarısı bu formülün görsel kanıtıdır: j ≤ i merdiveni ile j > i merdiveni birlikte n × (n + 1) dikdörtgenini doldurur; merdivenlerden biri bunun yarısıdır (H6'daki Gauss eşleştirmesinin geometrik hâli).

Tablo 7.1
H07_K02_yineleme_sayilari.c: sayaçla ölçülen yineleme sayıları (gerçek çıktı; her satırda formülle aynı)

N	A) j ≤ N	B) j ≤ i	C) j < i	D) a ≤ b ≤ c ≤ N
10	100	55	45	220
100	10 000	5 050	4 950	171 700
1000	1 000 000	500 500	499 500	167 167 000
formül	N^2	N(N+1)/2	N(N−1)/2	N(N+1)(N+2)/6

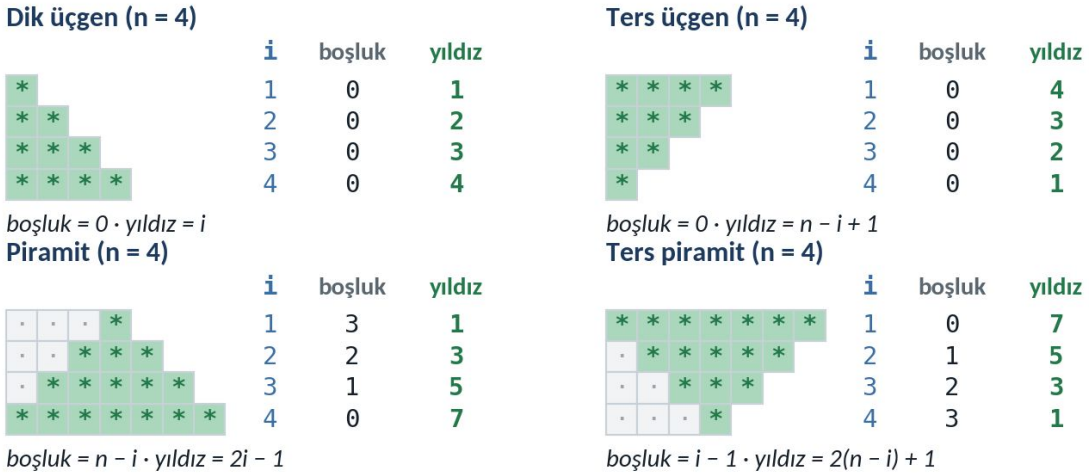
Tablonun asıl mesajı **büyüme hızıdır**: N on katına çıktığında tek döngü on kat, iki katlı iç içe döngü yüz kat, üç katlı döngü yaklaşık bin kat fazla yineleme yapar. Üçgen döngü dikdörtgenin yaklaşık yarısıdır ama o da N büyüdükçe karesel büyür. Bu hafta yineleme sayısını yalnızca **sayıyoruz** ve **ölçüyoruz**; biçimsel gösterimler (O, Ω, Θ) AP-2'nin konusudur.

3.3 Satır–sütun düşüncesi: desenleri formülle çizmek

Uçbirim yalnızca soldan sağa, yukarıdan aşağıya yazar; geri dönüp bir karakteri değiştiremeyiz. Bu yüzden bir deseni çizmek, onu **satır satır** üretmek demektir. Deneme-yanılma yerine dört adımlı bir yöntem izleyeceğiz:

1. Beklenen çıktıyı n = 4 gibi küçük bir değerle **kareli kâğıda** çizin; boşlukları da (·) işaretleyin.
2. Her satır için bir **satır tablosu** kurun: satır no `i`, baştaki boşluk sayısı, yıldız sayısı.
3. Sütunlardaki sayıları `i` ve `n` cinsinden bir **formüle** dönüştürün; formülü ilk ve son satırda sınayın.
4. Kodu yazın: dış döngü satırları dolaşır; gövdesinde formül başına **bir iç döngü** (önce boşluk, sonra yıldız) ve satır sonunda `printf("\n")`.

Şekil 7.2
*Dört desenin satır–boşluk–yıldız tabloları ve formülleri (n = 4); gri · boşluk, yeşil * yıldızdır*



Tablolar (Şekil 7.2) desenleri birbirine bağlar: **ters üçgen**, dik üçgende `i` yerine `n − i + 1` yazmaktır; **ters piramit**, piramitte `i` yerine `n − i + 1` yazmaktır (`n − (n − i + 1)` = `i − 1` boşluk, `2(n − i + 1) − 1` = `2(n − i) + 1` yıldız). **Elmas** ise iki yarıdan oluşur: üst yarı piramit (i = 1..n), alt yarı aynı formülle ama dış döngü **geriye** sayarak: `for (int i = n - 1; i >= 1; i--)` (`H07_K03_elmas.c`).

Satır sonundaki boşlukları yazmayız: piramidin sağında boşluk yoktur, çünkü satır son yıldızla biter. Tablodaki toplam yıldız sayısı da formülden gelir: piramitte 1 + 3 + … + (2n − 1) = n^2 (Örnek 4'ün gerçek çıktısı: n = 5 için 25), elmasta $n^2 + (n-1)^2$ (n = 4 için 16 + 9 = 25).

3.4 Problem çözme desenleri: H6'nın desenleri iç içe

H6'nın desen kartları bu hafta daha büyük problemlerin yapı taşlarına dönüşüyor. Bir problemi çözerken önce "hangi desen(ler)?" sorusunu sormak, boş sayfa korkusunu yener. Tablo 7.2 haftanın örneklerini desenlerine göre eşleştirir.

Tablo 7.2
Haftanın problem çözme desenleri ve H6'daki karşılıkları

Desen (H6'dan)	Bu haftaki biçimi	Örnek	H6'daki atası
Bayrak + break	Bölen arama: ilk bölen bulunca `asal = 0; break;`	6, 7, 11	`H06_13_sayi_tahmin.c` (bildi)
Toplayıcı (iç döngüde)	Bölen toplamı; her aday için **yeniden 0**	8 (mükemmel)	`H06_02_toplam_n.c`
Basamak ayrıştırma + çarpım	Basamak sayısı k, sonra basamak^k toplamı	8 (Armstrong)	`H06_09_basamak_islemleri.c`, `H06_06_us_alma.c`
Kayan değişkenler	Yalnızca son iki terim: `onceki`, `simdiki`	9 (Fibonacci)	`H06_11_ebob_ekok.c` (a, b kayar)
Ardışık bölme	`% 2` ve `/ 2`; kalanlar ters sırada	10 (ikili)	`H06_10_ters_palindrom.c`

Desen (H6'dan)	Bu haftaki biçimi	Örnek	H6'daki atası
Olay kontrollü döngü	Terim < ε olunca dur; deneme sayacı	11–13	`H06_14_collatz.c`, `H06_K06_desenler.c`
Girdi doğrulama döngüsü	Menünün içinde doğrulama döngüleri	14 (ATM)	`H06_12_gecerli_girdi.c`

İki uyarı her desene eşlik eder. (1) **İç döngünün değişkenleri dış döngünün her turunda yeniden başlatılmalıdır**: bölen toplamı her aday için 0'dan, bayrak her aday için 1'den başlar (Bölüm 5, HA1; Alıştırma 7.4). En güvenli yol, bu değişkenleri dış döngünün **içinde** bildirmektir. (2) **Her döngünün kendi sayacı olmalıdır** (HA2).

3.5 Asallık: naif test, √n sınırı ve tek bölenler

1'den büyük ve yalnızca 1'e ve kendisine bölünen tamsayıya **asal** denir; 0 ve 1 asal değildir, 2 tek çift asaldır. Tanımı doğrudan koda dökmek **naif testtir** (Örnek 6): 2'den n − 1'e kadar her `d` için `n % d == 0` mı diye bakarız; bir bölen bulunca bayrağı indirir ve `break` ile çıkarız. Asal bir n için bu, n − 2 bölme demektir; Örnek 6'nın gerçek çıktısı 97 için "95 bölme", 2 147 483 647 için "2 147 483 645 bölme"dir.

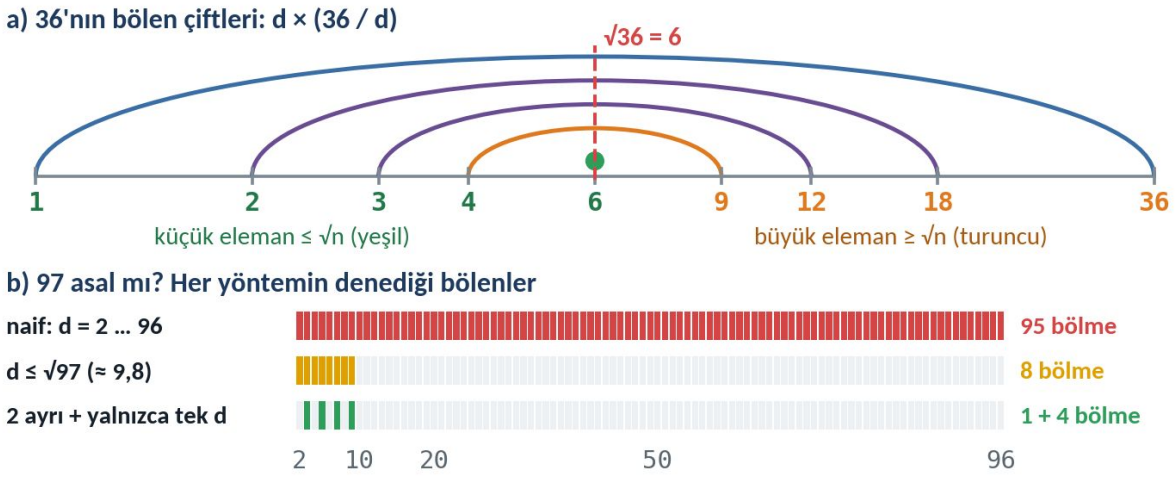
Neden √n yeter? n'nin her böleni d bir **çift** oluşturur: d × (n / d). Çiftin iki elemanı √n'nin iki yanındadır; ikisi birden √n'den büyük olsaydı çarpımları n'yi aşardı. Öyleyse √n'ye kadar bölen bulunamadıysa, ötesinde de yoktur. 36'nın çiftleri 6'nın etrafında aynalanır (Şekil 7.3a):

H07_K11_bolen_ciftleri.c · satır 22–27

```
22      for (d = 1; d <= n / d; d++) {  /* d² <= n, taşmasız yazımı */
23          if (n % d == 0) {
24              printf("  %d x %d\n", d, n / d);
25              cift++;
26          }
27      }
```

Gerçek çıktı (n = 36): 1 x 36, 2 x 18, 3 x 12, 4 x 9, 6 x 6; "5 bölen çifti; döngü d = 7 iken durdu (7 x 7 > 36)".

Şekil 7.3
a) 36'nın bölen çiftleri √36 = 6 etrafında aynalanır. b) 97 için üç yöntemin denediği bölenler



Koşulu `d * d <= n` diye de yazabilirdik; ama `d * d` büyük n'de **taşar**: n = 2 147 483 647 (INT_MAX) için d = 46 341'de 46 341² = 2 147 488 281 artık `int`'e sığmaz ve işaretli taşma tanımsız davranıştır

(Bölüm 5, HA6). `d <= n / d` aynı anlamdadır (tamsayı bölmesiyle $d \le \lfloor n/d \rfloor \Leftrightarrow d^2 \le n$) ama hiç taşmaz. İkinci iyileştirme **çift bölenleri atlamaktır**: n çift değilse hiçbir çift sayı onu bölemez. 2'yi ayrıca ele alıp `d = 3`'ten başlayarak `d += 2` ile yalnızca tek bölenleri deneriz. 97 için bölen adayları 95'ten (naif) 8'e (√n), sonra 1 çift denetimi + 4 tek bölene (3, 5, 7, 9) iner (Şekil 7.3b). Sınır durumları (n < 2, n = 2, çift n) döngüden **önce** ve bu sırayla ele alınmalıdır; aksi hâlde 1 asal, 2 asal değil çıkar (HA4).

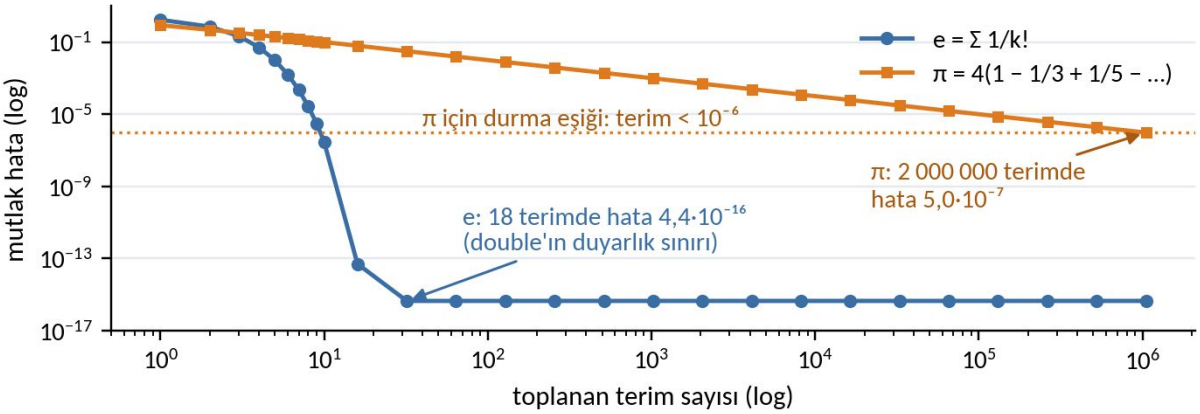
3.6 ε ile biten döngüler: seriler

Bazı döngülerin kaç tur döneceğini önceden bilemeyiz; döngüyü **istenen doğruluğa ulaşınca** durdururuz. Sonsuz bir serinin terimleri küçülüyorsa, terim önceden seçilmiş küçük bir **ε** (epsilon) eşiğinin altına inince toplamayı keseriz. Üç klasik seri:

- **e** = 1/0! + 1/1! + 1/2! + … Bir sonraki terim öncekinin k'ya bölümüdür (`terim /= k`): faktöriyeli ayrıca hesaplamaya gerek yoktur.
- **π** = 4 · (1 − 1/3 + 1/5 − 1/7 + …) (Leibniz serisi). İşaret her terimde değişir (`isaret = -isaret`); terimler çok yavaş küçülür.
- **sin x** = $x - x^3/3! + x^5/5! - \dots$ (Taylor serisi). Her terim bir öncekinden $-x^2 / ((2k + 2)(2k + 3))$ çarpanıyla elde edilir.

Örnek 12'nin gerçek çıktısına göre e, $\varepsilon = 10^{-15}$ eşiğiyle **18 terimde** $4{,}4 \times 10^{-16}$ hatayla hesaplanır; π ise $\varepsilon = 10^{-6}$ eşiğiyle **2 000 000 terim** gerektirir ve hatası yine de $5{,}0 \times 10^{-7}$'dir. Şekil 7.4, `H07_K09_yakinsama.c`'nin ölçtüğü hataları gösterir: e'nin hatası her yeni terimle çarpanlarla düşer, 16 terimde 5×10^{-14}'e iner ve Örnek 12'ye göre 18 terimde `double`'ın duyarlık sınırına (yaklaşık $4{,}4 \times 10^{-16}$) ulaşıp orada kalır; π'nin hatası ise terim sayısı iki katına çıktığında yalnızca yarıya iner.

Şekil 7.4
Terim sayısına karşı mutlak hata (log–log; H07_K09_yakinsama.c'nin gerçek çıktısı)



`H07_K10_sin_taylor.c` aynı yapıyla sin x'i 10^{-12} eşiğine kadar toplar ve `math.h`'deki `sin` ile karşılaştırır (gerçek çıktı: x = 1 için 7 terim, x = 10 için 23 terim; fark her ikisinde de 10^{-12} mertebesinin altında). İki ayrıntı önemlidir. (1) Terimler **kesirli** bölmeyle üretilmelidir: `1 / k` iki `int`'in bölümüdür ve k ≥ 2 için 0'dır (H4; HA5'te e = 2 çıkar). (2) Durma koşulu terimin **mutlak değerine** bakmalıdır (`fabs(terim) >= EPS`), çünkü işaret değişir; x = 0'da ilk terim zaten 0 olduğu için döngü hiç çalışmaz ve sonuç 0'dır (gerçek çıktı). `math.h` fonksiyonları için Linux'ta `-lm` bağlama seçeneği gerekir.

3.7 İç içe döngüden çıkış: bayrak, break ve goto

`break` yalnızca **içinde bulunduğu en içteki** döngüyü (ya da `switch`'i) bitirir. İç içe bir aramada "bulundu, her şeyi bırak" demek istediğimizde bu yetmez. `H07_K04_ic_ice_cikis.c`, $i^2 + j^2 = 325$ sağlayan **ilk** çifti ($i \le j \le 20$) üç yolla arar; 325'in üç gösterimi vardır ($1^2 + 18^2$, $6^2 + 17^2$, $10^2 + 15^2$):

H07_K04_ic_ice_cikis.c · satır 32–48

```c
    /* 2) Bayrak */
    int bulundu = 0;
    int bi = 0;
    int bj = 0;
    deneme = 0;
    for (int i = 1; i <= UST && !bulundu; i++) {  /* bayrağa da bak */
        for (int j = i; j <= UST; j++) {
            deneme++;
            if (i * i + j * j == HEDEF) {
                bulundu = 1;
                bi = i;
                bj = j;
                break;
            }
        }
    }
    printf("2) bayrak : %d^2 + %d^2, %d deneme\n", bi, bj, deneme);
```

Yalnız `break`'li sürüm üç çözümü de bulur ve **200 deneme** yapar; bayrak ya da `goto` ilk çözümde durur (**18 deneme**). Bayraklı çözümde dış döngünün koşulu `i <= UST && !bulundu` olur. H8'de dördüncü ve çoğu zaman en temiz yolu göreceğiz: aramayı bir **fonksiyona** koyup bulunca `return` etmek.

> MÜHENDİS GÖZÜYLE · **goto tartışması: "zararlı" mı?**
>
> Dijkstra (1968), denetimsiz `goto` kullanımının programın ilerleyişini izlemeyi zorlaştırdığını savunan kısa mektubunda bu deyimin "üst düzey programlama dillerinden kaldırılması" gerektiğini önerdi; yapısal programlama (sıra, seçim, tekrar; Böhm ve Jacopini, 1966; H2) bu tartışmayla yerleşti. Knuth (1974) ise `goto`'nun bazı durumlarda yapısal alternatiflerinden daha okunur olabileceğini örneklerle gösterdi.
>
> C'de `goto` hâlâ vardır ve iki dar kullanımı yaygın olarak kabul görür: **iç içe döngülerden tek adımda çıkış** ve bir fonksiyonun birden çok yerden çıkarken ortak **temizlik** yapması (kaynakları serbest bırakma). Linux çekirdeğinin kodlama kılavuzu ikinci kullanımı açıkça önerir; temizlik gerekmiyorsa doğrudan `return` edilmesini söyler (The Linux Kernel Documentation, t.y.). Bu derste kural: önce bayrak ya da fonksiyon + `return`; `goto` yalnızca ileri doğru, tek bir etikete ve iç içe döngüden çıkmak için.

3.8 Menü döngüsü: do-while + switch

Etkileşimli programların çoğu aynı iskeletle çalışır: menüyü göster, seçimi oku, seçime göre işi yap, kullanıcı "çıkış" diyene kadar başa dön. Menü **en az bir kez** gösterileceği için `do-while`, seçimlere göre dallanmak için `switch` (H5) doğal seçimdir (akış diyagramı sunumda ve web'de; `switch`, `do-while` ve `break` için ayrıca bkz. Deitel ve Deitel, 2022, Bölüm 4).

Aşağıdaki kesitin öncesinde döngü `do {` ile açılır ve `scanf` dönüşü denetlenir. Kritik nokta: `case` içindeki `break` **yalnızca** `switch`'ten çıkar; menüyü bitiren, döngü koşuludur (`secim != 0`, bir nöbetçi). `while (1)` ile yazıldığında çıkış denetimi `switch`'ten **önce** bir `break` ile yapılır (`H07_K13_menu_while1.c`; gerçek çıktı: "Çıkış: sayac = 2"). K12'nin gerçek çalıştırmasında (girdi `1 1 3 7 2 3 0`) 7 `default` dalına düşer ve menü yeniden gösterilir.

H07_K12_menu_do_while.c · satır 21–38

```c
        switch (secim) {
        case 1:
            sayac++;
            break;  /* yalnızca switch'ten çıkar */
        case 2:
            sayac--;
            break;
        case 3:
            printf("sayac = %d\n", sayac);
            break;
        case 0:
            printf("Güle güle.\n");
            break;
        default:
            printf("Geçersiz seçim: %d\n", secim);
            break;
        }
    } while (secim != 0);  /* menüyü bitiren: döngü koşulu */
```

3.9 Kaputun altında: "daha hızlı" iddiasını ölçmek

Yineleme sayısı algoritmanın **iş miktarıdır**; süre ise o işin belirli bir makinede, belirli bir derleyiciyle, o anki yük altında ne kadar sürdüğüdür. İkisi ilişkilidir ama aynı şey değildir. C standart kütüphanesindeki `clock()` (`time.h`), programın o ana kadar kullandığı **işlemci zamanını** `clock_t` türünden "tık" olarak döndürür; iki ölçümün farkını `CLOCKS_PER_SEC`'e bölerek saniye elde ederiz (ISO/IEC, 2018, 7.27.2.1):

H07_K05_clock_temel.c · satır 17–24

```c
    clock_t bas = clock();  /* başlangıç tıkı */
    long long toplam = 0;
    for (int i = 1; i <= 100000000; i++) {
        toplam += i % 7;
    }
    clock_t son = clock();  /* bitiş tıkı */

    double sure = (double)(son - bas) / CLOCKS_PER_SEC;
```

Gerçek çalıştırma: "geçen işlemci zamanı = 0.273 s (272767 tık)"; `time` komutuyla: `real 0m0.287s`, `user 0m0.280s` (`real` duvar saati, `user` işlemci zamanı). Bu makinede (glibc) `CLOCKS_PER_SEC` = 1 000 000'dur (POSIX bu değeri zorunlu kılar). **Taşınabilirlik:** Windows'taki Microsoft C çalışma zamanında (MinGW de onu kullanır) `clock()` program başlangıcından bu yana geçen **duvar saati** zamanını döndürür ve `CLOCKS_PER_SEC` = 1000'dir (Microsoft, t.y.).

Ölçüm ilkeleri. (1) **Tekrar edin, medyanı alın:** tek ölçüm gürültülüdür (işletim sistemi, önbellek, paylaşılan işlemci); çok kısa işleri bir döngüde R kez tekrarlayıp toplamı R'ye bölün. (2) **Sonucu kullanın:** derleyici sonucu kullanılmayan hesabı silebilir; sonucu yazdırın ya da toplayıcıyı `volatile` bildirin (aşağıda K07). (3) **Ortamı yazın:** işlemci, derleyici, seçenekler (`-O0` bu dersin varsayılanı, `-O2` iyileştirmeli), N ve tekrar sayısı. (4) **Oranı ve eğilimi yorumlayın:** mutlak süreler makineden makineye değişir; **yineleme sayısı ise değişmez**.

`H07_K07_olu_kod.c` aynı naif sayımı üç kez ölçer: A) sonuç kullanılmadan, B) sonuç yazdırılarak, C) `volatile` toplayıcıyla. `-O0` ile üçü de yaklaşık 0,108 saniye sürdü. `-O2` ile B ve C yaklaşık 0,086 saniye

sürerken A 0,000002 saniye "sürdü" (gerçek çıktılar, üç çalıştırmanın medyanı). Neden? `objdump -d` ile makine kodundaki bölme komutlarını (`idiv`) saydık: `-O0` sürümünde 3, `-O2` sürümünde 2 tane var. Derleyici, sonucu kullanılmayan A döngüsünü **tümüyle silmiştir**; bu, standardın izin verdiği bir iyileştirmedir (gözlenebilir davranış değişmez).

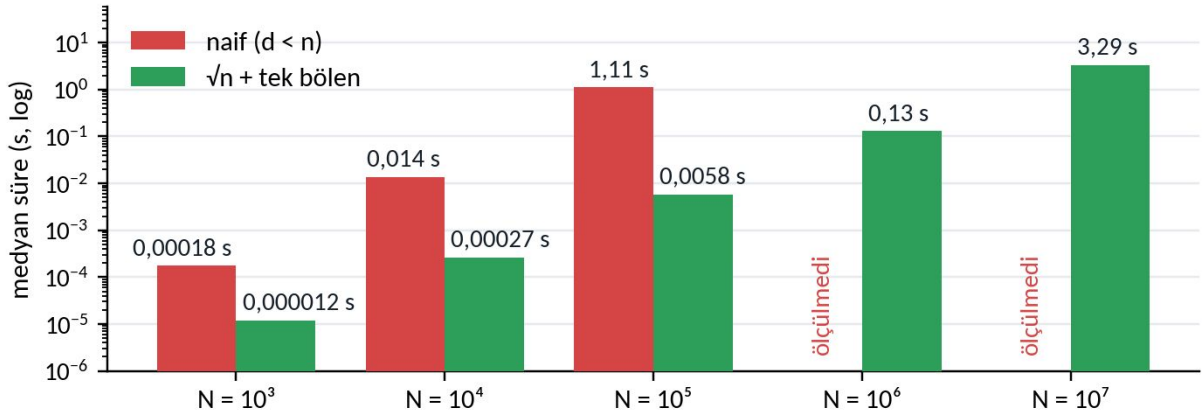
Ölçüm deneyi: naif ve iyileştirilmiş asallık

`H07_K06_olcum_deneyi.c` 0..N arasındaki asalları seçilen yöntemle sayar, işi program içinde **5 kez** tekrarlar ve her tekrarın `clock()` süresini yazar; program `-O0` ve `-O2` ile derlenip N = 10^3 … 10^7 için çalıştırıldı. Tabloda 5 tekrarın **medyanı** vardır. Ortam: Intel Xeon Processor @ 2.80GHz, 2 çekirdek, **paylaşımlı sanal makine**, GCC 13.3.0. Süreler göreli okunmalıdır; bölme sayıları sayaçla ölçülmüştür ve her makinede aynıdır.

Tablo 7.3
Ölçüm deneyi (gcc -O0, 5 tekrarın medyanı; gerçek ölçüm)

N	asal sayısı	naif: bölme	naif: süre (s)	iyileşt.: bölme	iyileşt.: süre (s)	oran: bölme / süre
10^3	168	78 022	0,000176	2 351	0,000012	33 / 15
10^4	1229	5 775 223	0,0137	55 958	0,000266	103 / 52
10^5	9592	455 189 150	1,1114	1 345 442	0,005778	338 / 192
10^6	78498	—	ölçülmedi	33 600 560	0,1312	—
10^7	664579	—	ölçülmedi	870 439 156	3,2945	—

Şekil 7.5
Naif ve iyileştirilmiş asallık testinin ölçülen süreleri (gcc -O0, 5 tekrarın medyanı; logaritmik eksen)



Okuma: N = 10^5'te bölme sayısı **338 kat**, süre yaklaşık **192 kat** azaldı (1,11 s → 5,8 ms). Oranların farkı, iyileştirilmiş yöntemde her sayı için döngü dışında da iş olmasından (sınır ve çiftlik denetimleri) gelir. Naif yöntemde N on katına çıkınca süre yaklaşık 81 katına çıktı (karesele yakın); bu eğilimle N = 10^6 bir dakikayı aşacağı için ölçülmedi.

Bölme sayısını süreye karşı çizersek (web sayfasındaki log–log grafik) süre bölme sayısıyla **orantılı** büyür; bölme başına yaklaşık 2,4 ns (naif) ve 3,8 ns (iyileştirilmiş, sayı başına iş dahil). `-O2` naif yöntemi yalnızca 1,32 kat hızlandırdı (zamanın çoğu bölme komutunun kendisinde geçer), iyileştirilmiş yöntemi 2,0 kat. Sonuç: **önce algoritma**, sonra derleyici.

4. Çözümlü Örnekler: Örnek Merdiveni

Örnekler kolaydan zora sıralıdır ve her basamak bir yeni öge ekler (dosyalar `kodlar/H07_01_…` – `H07_14_…`). Her biri `gcc -std=c17 -Wall -Wextra -pedantic -g` ile **sıfır uyarıyla** derlenir, AddressSanitizer ve UBSan ile de temiz çalışır. Çıktılar `kodlar/ciktilar/` altındaki gerçek çalıştırmalardır (girdiler uçbirimde yazılmış gibi görünür). Örnek 4, 11 ve 14 standart çözümlü örnek şablonunun tamamıyla işlenmiştir.

4.1 ★ basamaklar: dikdörtgen, üçgenler ve çarpım tablosu (Örnek 1–3)

Örnek 1 ★ — n × m yıldız dikdörtgeni

Problem. Satır ve sütun sayısını okuyup o boyutta bir yıldız dikdörtgeni çizin ve iç gövdenin kaç kez çalıştığını yazın.

H07_01_dikdortgen.c · satır 19–27

```
19      int yineleme = 0;                       /* iç gövdenin kaç kez çalıştığı */
20      for (int i = 1; i <= satir; i++) {      /* dış döngü: her tur bir SATIR */
21          for (int j = 1; j <= sutun; j++) {  /* iç döngü: her tur bir SÜTUN */
22              printf("*");
23              yineleme++;
24          }
25          printf("\n");  /* satır bitti: alt satıra geç */
26      }
27      printf("Toplam yineleme: %d (%d x %d)\n", yineleme, satir, sutun);
```

`printf("\n")` iç döngünün **dışında**, dış döngünün içindedir: her satırın sonunda bir kez çalışır. İçeri taşınırsa her yıldız ayrı satıra düşer; dış döngünün dışına taşınırsa tüm yıldızlar tek satırda çıkar. Girdi “3 5” için gerçek çıktı üç satır `*****` ve “Toplam yineleme: 15 (3 x 5)”. **Değiştir:** dikdörtgenin yalnızca kenarlarını çizin (iç hücreler boşluk); hangi koşul `i` ve `j` ile yazılır?

Örnek 2 ★ — Dik üçgen ve ters üçgen

Problem. n satırlı dik üçgen ve ters üçgen çizin. İç döngünün sınırı artık sabit değil, dış değişkene bağlıdır (Şekil 7.2).

Dik üçgenin iç döngüsü `for (int j = 1; j <= i; j++)` biçimindedir: i. satırda i kez döner; toplam 1 + 2 + … + n = n(n + 1)/2 (gerçek çıktıda 15). Ters üçgen aynı programda `j <= n - i + 1` sınırıyla çizilir. **Değiştir:** dik üçgeni sağa yaslayın (önce n − i boşluk).

Örnek 3 ★ — 10 × 10 biçimli çarpım tablosu

H6’da (`H06_04_carpim_tablosu.c`) tek bir sayının çarpım tablosunu tek döngüyle yazmıştık; şimdi o döngüyü bir dış döngünün içine koyarak tablonun tamamını üretiyoruz. Başlık satırı ve ayırıcı çizgi de birer döngüdür; `%4d` alan genişliği (H3) sütunları hizalar.

Gövde, Örnek 1’in dikdörtgeniyle aynı iskelettir; yıldız yerine `printf("%4d", i * j);` yazılır (kod: `kodlar/H07_03_carpim_tablosu.c`). Gerçek çıktının son satırı: `10 | 10 20 30 40 50 60 70 80 90 100`. 10 × 10 = 100 hücre, iç gövdenin 100 yinelemesidir.

4.2 Amiral örnek — Örnek 4 ★★: piramit (boşluk + yıldız)

1. Problem. Yüksekliği n (1–30) olan, ortalanmış bir yıldız piramidi çizin ve toplam yıldız sayısını yazın. Bu, “satır başına formül çıkarma” becerisinin tipik sınavıdır.

Tablo 7.4
Örnek 4'ün girdi–işlem–çıktı çözümlemesi

GİRDİ	İŞLEM	ÇIKTI
• n: piramidin yüksekliği (tamsayı, 1–30)	• Her satır i için: n − i boşluk, 2i − 1 yıldız • Yıldızları say (toplayıcı/sayaç)	• n satırlık piramit • Toplam yıldız sayısı (= n^2) • Geçersiz n için ileti

2. Çözümleme ve sınırlar. n = 1 tek yıldızdır (0 boşluk). Üst sınır (30) satır uzunluğunu (2n − 1 = 59) uçbirim genişliğinde tutmak içindir. Sayı olmayan ya da aralık dışı girdi reddedilir. **3. Algoritma.** Satır tablosu (Şekil 7.2, piramit): i = 1 → 3 boşluk, 1 yıldız; i = 4 → 0 boşluk, 7 yıldız (n = 4). Formül: boşluk = n − i, yıldız = 2i − 1. Sözde kod: *her i = 1..n için: (n − i) kez boşluk yaz; (2i − 1) kez yıldız yaz; satır sonu*.

4. C kodu.

H07_04_piramit.c · satır 9–33

```c
#include <stdio.h>

int main(void)
{
    int n;
    printf("Piramidin yüksekliği n (1-30): ");
    if (scanf("%d", &n) != 1 || n < 1 || n > 30) {
        printf("Geçersiz girdi.\n");
        return 1;
    }

    int toplam = 0;
    for (int i = 1; i <= n; i++) {
        for (int b = 1; b <= n - i; b++) {  /* önce n - i boşluk */
            printf(" ");
        }
        for (int y = 1; y <= 2 * i - 1; y++) {  /* sonra 2i - 1 yıldız */
            printf("*");
            toplam++;
        }
        printf("\n");
    }
    printf("Toplam %d yıldız (n² = %d)\n", toplam, n * n);
    return 0;
}
```

5. Satır satır. Satır 21: dış döngü satırları sayar. Satır 22 ve 25: dış döngünün gövdesinde **art arda iki** iç döngü vardır; ikisi de dış değişken `i`'ye bağlıdır ama birbirlerinden bağımsızdır (biri bitince öteki başlar; iç içe değil, **sıralı**). Satır 27: sayaç yalnızca yıldız döngüsünde artar. Satır 29: satır sonu, iki iç döngüden sonra.

Tablo 7.5
Örnek 4 izleme tablosu (n = 3)

i	b-döngüsü (boşluk)	y-döngüsü (yıldız)	yazılan satır (· = boşluk)	toplam
1	2	1	`··*`	1
2	1	3	`·***`	4
3	0	5	`*****`	9
4	—	—	`koşul 4 <= 3 yanlış: döngü biter`	**9**

7–8. Örnek çalıştırma ve test tablosu (gerçek çıktılar; n = 5 için piramidin kendisi Şekil 7.2'deki n = 4 piramidinin bir satır büyüğüdür):

Tablo 7.6
Örnek 4 test tablosu

Test	Girdi	Beklenen	Gerçek çıktı (son satır)	Sonuç
Normal	5	5 satır, 25 yıldız	Toplam 25 yıldız ($n^2 = 25$)	✓
Sınır: en küçük	1	tek yıldız, 0 boşluk	Toplam 1 yıldız ($n^2 = 1$)	✓
Normal (izleme)	3	9 yıldız	Toplam 9 yıldız ($n^2 = 9$)	✓

9. Değiştir–genişlet. (a) Ters piramit (Alıştırma 7.3'ün doğru sürümü). (b) Elmas (Bölüm 3.3, `H07_K03_elmas.c`). (c) Yıldız yerine satır numarasını yazın (1, 222, 33333 …). (d) İçi boş piramit: yalnızca satırın ilk ve son yıldızı ile son satırın tamamı.

4.3 Örnek 5 ★★ — Floyd üçgeni ve sayı üçgeni

Problem. Floyd üçgeninde sayılar 1'den başlar ve satırlar boyunca **kesintisiz** artar (1 / 2 3 / 4 5 6 …). Aynı programın ikinci deseni, her satırda yeniden 1'den başlayan sayı üçgenidir. Fark, desen içindeki sayacın **nerede** tutulduğudur.

H07_05_floyd_ucgeni.c · satır 21–29

```
21      int sayi = 1;  /* satırlar boyunca süren sayaç */
22      for (int i = 1; i <= n; i++) {
23          for (int j = 1; j <= i; j++) {
24              printf("%4d", sayi);
25              sayi++;
26          }
27          printf("\n");
28      }
29      printf("Son sayı: %d = n(n+1)/2\n", sayi - 1);
```

`sayi` dış döngüden **önce** bir kez başlatılır ve hiç sıfırlanmaz: üçgendeki son sayı yineleme sayısının ta kendisidir, n(n + 1)/2 (n = 5 için gerçek çıktı: "Son sayı: 15 = n(n+1)/2"). Sayı üçgeninde ise yazılan `j`'dir ve `j` her satırda `for` başlığında yeniden 1 olur. Bu ikilik, Hata Avcısı'ndaki "sıfırlanması gereken / gerekmeyen değişken" sorusunun olumlu yüzüdür: Floyd'da sürmesi **istenen** bir sayaç vardır.

4.4 Asallık: naif test ve 1–1000 arası asallar (Örnek 6–7)

Örnek 6 ★★ — Naif asallık testi

Problem. Okunan tamsayının asal olup olmadığını söyleyin; değilse bir bölen çiftini, her durumda kaç bölme denendiğini yazın. Kod, Bölüm 3.5'teki tanımın doğrudan karşılığıdır; H8'de bu blok bir `is_prime` fonksiyonuna taşınacak.

H07_06_asal_naif.c · satır 20–34

```
20      int asal = 1;   /* bayrak: aksi kanıtlanana dek asal */
21      int bolme = 0;  /* kaç bölme denendi */
22      int bolen = 0;  /* bulunan ilk bölen */
23      if (n < 2) {
24          asal = 0;  /* 0, 1 ve negatifler asal değildir */
25      } else {
26          for (int d = 2; d < n; d++) {
27              bolme++;
28              if (n % d == 0) {
29                  asal = 0;
30                  bolen = d;
31                  break;  /* bir bölen yeter: aramayı bitir */
32              }
33          }
34      }
```

Gerçek çıktılar: "97 asaldır (95 bölme denendi).", "91 asal değildir: 7 x 13 (6 bölme denendi).", "2 asaldır (0 bölme denendi).". Sınırlar: n < 2 (0, 1, negatifler) döngüye girmeden asal değil sayılır; n = 2'de döngü koşulu (2 < 2) ilk denetimde yanlıştır, bayrak 1 kalır ve 2 doğru biçimde asal çıkar (0 bölme). Naif testin bedelini büyük bir asal gösterir: "1000003 asaldır (1000001 bölme denendi)." ve en büyük `int` için "2147483647 asaldır (2147483645 bölme denendi)." (bu çalıştırma -O0 ile birkaç saniye sürdü).

Örnek 7 ★★ — 1–1000 arasındaki asallar

Örnek 6'nın testini bir dış döngünün içine koyunca (`H07_07_asallar_1_1000.c`) **iç içe döngüyle listeleme** elde ederiz: dış döngü adayları, iç döngü bölenleri dolaşır. Bayrak (`asal = 1`) dış döngünün **içinde** bildirilir; böylece her aday için yeniden 1 olur (Alıştırma 7.4'teki hata tam olarak bunun unutulmasıdır).

Gerçek çıktının ilk satırı `2  3  5  7  11  13  17  19  23  29`, son satırı "Toplam 168 asal; iç döngü 78022 kez döndü.". 1–1000 arasında **168** asal vardır; iç döngü toplam **78 022** kez döndü. Bileşik sayılar küçük bir bölende hemen `break` ile çıkar; maliyetin neredeyse tamamını asallar ödetir (her asal p için p − 2 bölme). İyileştirilmiş testle aynı liste 2 351 bölme gerektirir (Örnek 11'in N = 1000 çıktısı). Satır sonu, **sayaç** deseninin ikinci kullanımıdır: her 10 asalda bir `\n`.

4.5 Örnek 8 ★★ — Mükemmel sayılar ve Armstrong sayıları

Mükemmel sayı, kendisi hariç pozitif bölenlerinin toplamı kendisine eşit olan sayıdır (6 = 1 + 2 + 3). **Armstrong sayısı** (narsist sayı), k basamaklı olup basamaklarının k. kuvvetleri toplamı kendisine eşit olan sayıdır ($153 = 1^3 + 5^3 + 3^3$). İlki iç içe döngüde **toplayıcı** kullanır (her aday için `int toplam = 0;` dış döngünün içinde, sonra `for (int d = 1; d <= n / 2; d++)` ile öz bölenler toplanır); ikincisi **üç katlı** iç içe döngüde basamak ayrıştırma ve çarpım biriktiricidir:

H07_08_mukemmel_armstrong.c · satır 41–54

```
41          int k = 0;  /* basamak sayısı */
42          for (int s = n; s > 0; s /= 10) {
43              k++;
44          }
45          int toplam = 0;
46          for (int s = n; s > 0; s /= 10) {  /* her basamak için */
47              int basamak = s % 10;
48              int kuvvet = 1;
49              for (int e = 1; e <= k; e++) {  /* basamak^k: tekrarlı çarpma */
50                  kuvvet *= basamak;
51              }
52              toplam += kuvvet;
53          }
54          if (toplam == n) {
```

10 000'e kadar dört mükemmel sayı vardır (6, 28, 496, 8128) ve 1–99 999 arasında 19 Armstrong sayısı (tek basamaklıların hepsi dahil) bulunur; gerçek çıktı: "8128 = 1 + 2 + 4 + 8 + 16 + 32 + 64 + 127 + 254 + 508 + 1016 + 2032 + 4064" ve Armstrong satırı "1 2 3 4 5 6 7 8 9 153 370 371 407 1634 8208 9474 54748 92727 93084". Öz bölenleri n/2'ye kadar aramak yeter, çünkü n'den küçük en büyük bölen en çok n/2'dir. Armstrong'da her aday için iki iç döngü (basamak sayma, sonra toplama) ve onun içinde üçüncü bir döngü (kuvvet) vardır; `pow` (H4) `double` döndürdüğü için tamsayı kuvvetini tekrarlı çarpmayla (H6, Örnek 6) hesaplıyoruz.

4.6 Kayan değişkenler ve ardışık bölme (Örnek 9–10)

Örnek 9 ★★ — Fibonacci dizisi (yinelemeli)

F(0) = 0, F(1) = 1 ve k ≥ 2 için F(k) = F(k − 1) + F(k − 2). Diziyi saklamadan üretmek için yalnızca **son iki terimi** tutarız; her turda bu iki değişkenlik **pencere** bir adım sağa kayar.

H07_09_fibonacci.c · satır 30–37

```
30      int k = 1;  /* simdiki = F(k) */
31      while (simdiki <= INT_MAX - onceki) {
32          int sonraki = onceki + simdiki;
33          onceki = simdiki;
34          simdiki = sonraki;
35          k++;
36      }
37      printf("int'e sığan son terim : F(%d) = %d\n", k, simdiki);
```

Atama **sırası** önemlidir: `onceki = simdiki;` önce yapılmazsa eski değer kaybolur (H3'teki takas sorunu). Döngü toplamadan **önce** `simdiki <= INT_MAX - onceki` denetimini yapar; böylece hiçbir zaman taşan bir toplama yapılmaz. Gerçek çıktıya göre 32 bitlik `int`'e sığan son terim **F(46) = 1 836 311 903**'tür; F(47) = 2 971 215 073 sığmaz. 64 bitlik `unsigned long long` **F(93)**'e kadar yeter; F(94) için işaretsiz toplam 2^{64} modunda sarar (işaretsiz tamsayıda bu tanımlıdır).

> **HATA AVCISI · int ile F(47): tanımsız davranış**
>
> Aynı toplamayı denetimsiz `int` ile yapan `H07_K08_fib_int_ub.c` (BU DOSYA KASITLI TANIMSIZ DAVRANIŞ İÇERİR) derleyici uyarısı üretmez, çünkü değerler çalışma anında oluşur. **İşaretli tamsayı taşması tanımsız davranıştır**: standart, programın ne yapacağını belirlemez. Sorunu çalışma anında yakalamak için `-fsanitize=undefined` ile derleriz.
>
> UBSan'ın gerçek iletisi: `H07_K08_fib_int_ub.c:15:13: runtime error: signed integer overflow: 1134903170 + 1836311903 cannot be represented in type 'int'`. İletiden sonra ekrana yazılan F(47) satırı güvenilir bir değer değildir; başka derleyici ya da seçenekle başka bir şey olabilir.

Örnek 10 ★★ — Onluk tabandan ikili tabana

H1'de ikili sayıları elle çevirmiştik. Algoritma **ardışık 2'ye bölmedir**: kalan (`n % 2`) en sağdaki biti verir, bölüm (`n / 2`) kalan bitleri taşır; bölüm 0 olunca dururuz. `do-while` kullanırız ki n = 0 da bir bit ("0") üretsin.

H07_10_onluk_ikili.c · satır 24–29

```
24      int sayi = n;
25      do {  /* n = 0 için de bir kez: "0" */
26          printf("%d", sayi % 2);
27          sayi /= 2;
28      } while (sayi > 0);
29      printf("   <- bitler TERS sırada!\n");
```

Kalanlar **en az anlamlı bitten** başlayarak üretilir ama sayı **en anlamlı bitten** başlayarak yazılır: 13 için ekrana 1011 çıkar, doğrusu 1101 (Şekil sunumda ve web'de). Bu **tersten yazdırma sorunudur**; doğal çözümleri **H9'da özyineleme** (önce bölümü yazdır, sonra kalanı) ve **H10'da dizidir**. Dizisiz **geçici çözüm**: n'yi aşmayan en büyük 2 kuvvetinden aşağı inmek (programın ikinci bölümü; 2026 için "2026 (onluk) = 11111101010 (ikili)"). Koşul `kuvvet <= n / 2` taşmaz (`kuvvet * 2 <= n` taşabilirdi).

4.7 Amiral örnek — Örnek 11 ★★★: iyileştirilmiş asallık ve süre ölçümü

1. Problem. 0..N arasındaki asalları hem naif yöntemle (Örnek 6) hem iyileştirilmiş yöntemle sayın; iki yöntemin aynı sonucu verdiğini doğrulayın, her birinin bölme sayısını ve süresini yazın. Program, Büyük Soru'nun iki yarısını birden yanıtlar: daha hızlı nasıl anlarız ve bunu nasıl ölçeriz?

Tablo 7.7
Örnek 11'in girdi–işlem–çıktı çözümlemesi

GİRDİ	İŞLEM	ÇIKTI
• N: üst sınır (2–200 000)	• Naif: d = 2 .. n−1 • İyileştirilmiş: n<2 → değil; 2 → asal; çift → değil; tek d, d ≤ n/d • Her yöntem için sayaç ve clock()	• Asal sayısı (iki yöntemde aynı) • Bölme sayıları ve süreler • Bölme oranı

2. Çözümleme. Dış döngü n = 0'dan başlar ki sınır durumları (0, 1, 2) da sınansın. Bölme sayısı `long long` tutulur: N = 10^5'te naif sayım 4,5 × 10^8'i bulur. Üst sınır (200 000) naif yöntemin birkaç saniyede bitmesi içindir. **3. Algoritma.** İyileştirilmiş test sırasıyla: (1) n < 2 → asal değil; (2) n = 2 → asal; (3) n çift → asal değil; (4) d = 3, 5, 7, … için d ≤ n / d oldukça n % d'ye bak.

4. C kodu (iyileştirilmiş bölüm; naif bölüm Örnek 6'nın döngüsüdür):

H07_11_asal_karekok_sure.c · satır 45–68

```
45      /* ---- 2) İyileştirilmiş: √n sınırı ve yalnızca tek bölenler ---- */
46      long long iyi_bolme = 0;
47      int iyi_adet = 0;
48      bas = clock();
49      for (int n = 0; n <= ust; n++) {
50          int asal = 1;
51          if (n < 2) {
52              asal = 0;  /* 0 ve 1 asal değil */
53          } else if (n == 2) {
54              asal = 1;  /* tek çift asal */
55          } else if (n % 2 == 0) {
56              asal = 0;  /* 2'den büyük çiftler */
57          } else {
58              for (int d = 3; d <= n / d; d += 2) {  /* d <= √n, d tek */
59                  iyi_bolme++;
60                  if (n % d == 0) {
61                      asal = 0;
62                      break;
63                  }
64              }
65          }
66          iyi_adet += asal;
67      }
68      double iyi_sn = (double)(clock() - bas) / CLOCKS_PER_SEC;
```

5. Satır satır. Satır 51–56: sınır durumları **döngüden önce** ve bu sırayla; sıra değişirse 2 "çift olduğu için" asal değil çıkar (HA4). Satır 58: yalnızca tek bölenler ve taşmasız √n sınırı. Satır 27 ve 68: ölçülen bölgenin başı ve sonu; ekrana yazdırma ölçümün **dışında** kalır.

Tablo 7.8
Örnek 11 izleme tablosu: n = 97 için iyileştirilmiş testin döngüsü

Denetim	d	n / d	d <= n / d	n % d
1	3	32	doğru	1
2	5	19	doğru	2
3	7	13	doğru	6
4	9	10	doğru	7
5	11	8	**yanlış: dur**	–

97 için döngü 4 bölmede (3, 5, 7, 9) biter; hiçbir kalan 0 değildir, asal. Naif test 95 bölme yapmıştı. **7–8. Örnek çalıştırma ve test tablosu** (gerçek çıktılar; N = 100 000 için tam çıktı Bölüm 3.9'daki deneyle aynı yapıdadır):

Testler (gerçek çıktılar; iki yöntem her N'de aynı asal sayısını verdi): N = 1000: 168 asal, bölme 78 022 → 2 351; N = 10000: 1229 asal, bölme 5 775 223 → 55 958; N = 100000: 9592 asal, bölme 455 189 150 → 1 345 442; geçersiz girdi "1" için "Geçersiz girdi.". Süreler her çalıştırmada biraz değişir, sayılar değişmez.

9. Değiştir–genişlet. (a) Yalnızca 6k ± 1 biçimindeki bölenleri deneyin (5, 7, 11, 13, …): bölme sayısı ne kadar azalır? (b) Ölçümü R kez tekrarlayıp en kısa süreyi yazın (`H07_K06_olcum_deneyi.c`). (c) Atölye: ölçüm deneyini kendi bilgisayarınızda yapın, tabloyu ve grafiği Bölüm 3.9 ile karşılaştırın.

4.8 Seriler, kaba kuvvet ve arama uzayı (Örnek 12–13)

Örnek 12 ★★★ — e ve π yaklaşımı

Problem. e'yi Σ 1/k! ile 10^{-15} eşiğine, π'yi Leibniz serisiyle 10^{-6} eşiğine kadar hesaplayın; terim sayısını ve gerçek değerle farkı yazın (Bölüm 3.6).

Kod (`kodlar/H07_12_e_pi_seri.c`) iki ε döngüsünden oluşur: e için `terim /= k`, π için her turda `isaret = -isaret;` ve `buyukluk = 4.0 / (2 * t + 1);`. Gerçek çıktı Bölüm 3.6'da verildi (e: 18 terim; π: 2 000 000 terim). Karşılaştırma değerleri adlandırılmış sabitlerdir (`E_GERCEK`, `PI_GERCEK`); `M_PI` C standardında tanımlı değildir (POSIX eklentisidir), bu yüzden kullanmadık. Mutlak değer `fabs` yerine bir `if` ile alınır; program `math.h` gerektirmez. `4.0 / (2 * t + 1)` ifadesinde `4.0` bölmeyi kesirli yapar (HA5).

Örnek 13 ★★★ — Pisagor üçlüleri: kaba kuvvet ve arama uzayı

Problem. $a^2 + b^2 = c^2$ sağlayan tüm $a \le b \le c \le N$ üçlülerini bulun. **Kaba kuvvet** (brute force), aday çözümlerin tamamını sistematik olarak dener; adayların kümesine **arama uzayı** denir. Kaba kuvvet sürümü üç katlı bir döngüdür (`a` 1..N, `b` a..N, `c` b..N) ve arama uzayı N(N + 1)(N + 2)/6 üçlüdür (Tablo 7.1, D sütunu). Daraltılmış sürüm:

`H07_13_pisagor_ucluleri.c · satır 42–55`

```
42      for (int a = 1; a <= ust; a++) {
43          for (int b = a; b <= ust; b++) {
44              int hedef = a * a + b * b;
45              if (hedef > ust * ust) {
46                  break;  /* daha büyük b'ler de aşar */
47              }
48              for (int c = b + 1; c * c <= hedef; c++) {
49                  deneme2++;
50                  if (c * c == hedef) {
51                      bulunan2++;
52                  }
53              }
54          }
55      }
```

Daraltma. (1) $a \ge 1$ olduğundan $c > b$'dir: c, b + 1'den başlar. (2) c^2 hedefi ($a^2 + b^2$) aşınca daha büyük c'ler de aşar: iç döngü `c * c <= hedef` ile erken biter. (3) $a^2 + b^2 > N^2$ ise hiçbir $c \le N$ yetmez ve b büyüdükçe durum kötüleşir: `break` ile b döngüsünden çıkılır. Sonuç aynıdır, deneme sayısı N = 30'da 4 960'tan 615'e, N = 500'de 20 958 500'den 3 228 813'e iner (gerçek çıktı). Çarpımlar (`c * c` en çok 500^2 = 250 000) `int`'e rahatça sığar. Bu, Deep Blue'nun ölçekli olarak yaptığının küçük bir modelidir: arama uzayını **budamak**.

`H07_K14_oyun_agaci.c` bu büyümeyi hesaplar (çarpım biriktirici): 3 hamle çifti 10^9 konumdur ve 126 milyon konum/s hızla yaklaşık 8 saniye sürer; 6 hamle çifti 10^{18} konumdur ve yaklaşık 250 yıl sürerdi. Kaba kuvvet, arama uzayı iç içe döngüler gibi **çarpılarak** büyüdüğünde tek başına yetmez; Deep Blue'nun seçici araması ve AP-2'de göreceğimiz budama teknikleri bu yüzden vardır.

4.9 Amiral örnek — Örnek 14 ★★★: menü güdümlü ATM simülasyonu (ara proje)

1. Problem. Bir ATM'nin metin tabanlı benzetimini yazın: kullanıcı en çok 3 denemede PIN'ini girer (3 hatada kart bloke edilir); doğru PIN'den sonra menüden bakiye sorgular, para yatırır, para çeker ya da çıkar. Her girdi doğrulanmalı: sayı olmayan girdi programı kilitlememeli, kurallara uymayan tutar yeniden sorulmalı. Bu örnek ara projenin A seçeneğinin referans çözümüdür (Bölüm 7.2) ve H5–H7'nin bütün yapılarını birleştirir.

Tablo 7.9
Örnek 14'ün girdi–işlem–çıktı çözümlemesi

GİRDİ	İŞLEM	ÇIKTI
• PIN (en çok 3 deneme) • Menü seçimi (1–4) • Tutarlar (tamsayı TL)	• PIN: sayaç kontrollü for + bayrak + break • Menü: do-while + switch • Yatırma: 1..10 000 olana dek sor • Çekme: 10'un katı, ≤ bakiye; 0 = iptal	• Bakiye ve işlem iletileri • 3 hatalı PIN: "Kartınız bloke…", çıkış kodu 2 • EOF: "Girdi sona erdi.", çıkış kodu 1

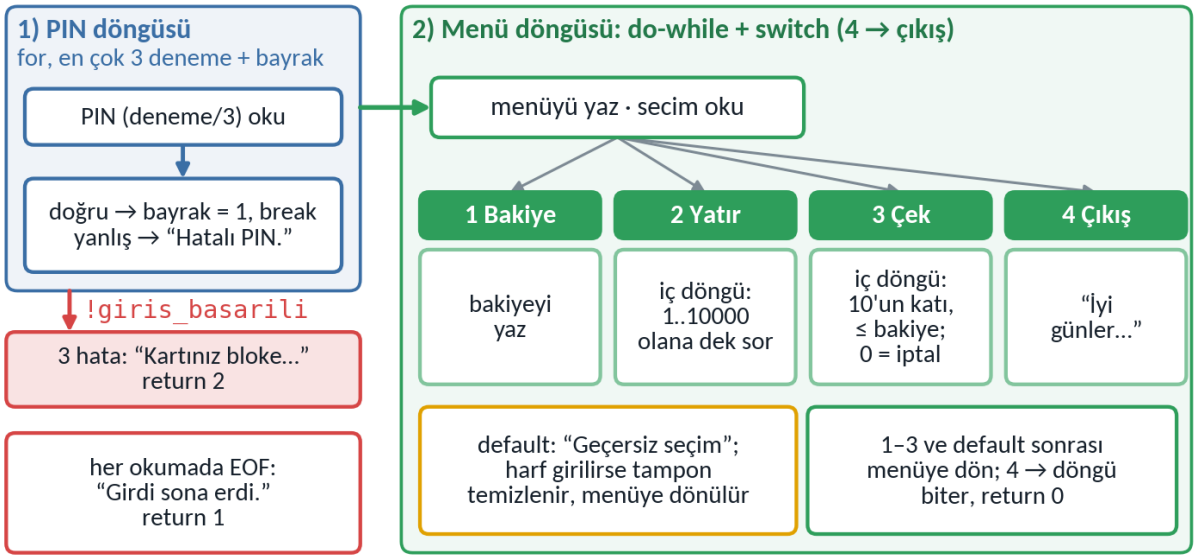
2. Kurallar ve kısıtlar. PIN 1234 (eğitsel örnek: gerçek sistemlerde PIN asla kodda tutulmaz; Bölüm 6). Başlangıç bakiyesi 1000 TL; tutarlar tamsayı TL'dir (kuruş yok, kayan nokta yuvarlama sorunlarından kaçınmak için). Çıkış kodları: 0 normal, 1 girdi yarıda bitti, 2 kart bloke. **3. Algoritma:** iki ardışık döngü; birincinin içinde doğrulama, ikincinin her `case`'inde ayrı bir doğrulama döngüsü (Şekil 7.6).

4. C kodu (tamamı 144 satır: `kodlar/H07_14_atm_menu.c`; burada yalnızca PIN döngüsü). Bu döngü, sayaç kontrollü bir `for` ile bayrağın birleşimidir:

H07_14_atm_menu.c · satır 29–50

```
29      for (int deneme = 1; deneme <= AZAMI_DENEME; deneme++) {
30          printf("PIN (%d/%d)           : ", deneme, AZAMI_DENEME);
31          okunan = scanf("%d", &pin);
32          if (okunan == EOF) {
33              printf("\nGirdi sona erdi.\n");
34              return 1;
35          }
36          if (okunan == 0) {
37              while ((c = getchar()) != '\n' && c != EOF) {
38                  /* satırın kalanını at */
39              }
40              pin = -1;
41          }
42          if (pin == DOGRU_PIN) {
43              giris_basarili = 1;
44              break;                          /* for döngüsünden çık */
45          }
46          printf("  Hatalı PIN.\n");
47      }
48      if (!giris_basarili) {
49          printf("Kartınız bloke edildi. Lütfen şubenize başvurun.\n");
50          return 2;
```

Şekil 7.6
ATM'nin denetim yapısı: PIN döngüsü (for + bayrak) ve menü döngüsü (do-while + switch); case'lerin içinde doğrulama döngüleri



Para çekme `case`'i (satır 97–133) menü döngüsünün içindeki ikinci bir doğrulama döngüsüdür: `do { … } while (!gecerli);` (satır 103–125). Tutar 0 ise vazgeçilir (`gecerli = 1`), negatif ya da 10'un katı değilse "Geçersiz...", bakiyeyi aşıyorsa "Yetersiz bakiye..." yazılıp yeniden sorulur; bakiye 10 TL'nin altındaysa döngüye hiç girilmeden `break` ile menüye dönülür.

Döngüden sonra tutar uygulanır (0 ise "İşlem iptal edildi.") ve `case` bir `break` ile biter. Menü döngüsü `} while (secim != 4);` ile kapanır: koşul yalnızca 4'te yanlıştır.

5. Satır satır. Satır 44: bu `break` bir `for`'un içindedir ve onu bitirir; satır 101'deki `break` ise bir `switch`'in içindedir ve **yalnızca** `switch`'ten çıkar (çekilebilir tutar yok: menüye dönülür). Sayı olmayan her girdide aynı üç adım tekrar eder: `scanf` dönüşünü denetle, `EOF`'ta çık, 0'da satırın kalanını `getchar` ile at ve değişkene geçersiz bir değer ver ki doğrulama döngüsü yeniden sorsun. Bu blok programda **dört kez** yazıldı; H8'de `tamsayi_oku()` gibi tek bir fonksiyona dönüşecek (Örnek 13 / Mini-OBS v0.8).

7–8. Örnek çalıştırmalar ve test tablosu. Beş senaryonun tam uçbirim dökümleri `kodlar/ciktilar/H07_14_atm_menu.txt` dosyasındadır; özetleri:

Tablo 7.10
Örnek 14 test senaryoları (gerçek çalıştırma; tam dökümler ciktilar/ altında)

Senaryo	Girdiler (boruyla)	Son ileti (gerçek)	Kod
normal akış	1111 1234 1 3 250 2 100 4	İyi günler dileriz.	0
üç hatalı PIN	1111 2222 abc	Kartınız bloke edildi. Lütfen şubenize başvurun.	2
geçersiz girdiler, yetersiz bakiye, iptal	1234 7 x 2 0 20000 500 3 15 -10 5000 0 1 4	İyi günler dileriz.	0
bakiyenin tamamını çekme; sonra çekilebilir tutar yok	1234 3 1000 3 4	İyi günler dileriz.	0
girdi yarıda biter (Ctrl+D)	1234 2	Girdi sona erdi.	1

9. Değiştir–genişlet. (a) Günlük çekme sınırı (ör. 2000 TL) ekleyin: hangi değişken, hangi desen? (b) İşlem sayacı ve "son işlem" özeti. (c) PIN değiştirme menüsü (eski PIN doğrulandıktan sonra yeni PIN iki kez girilip eşleşmeli). Bunlar ara projenin "ek özellik" puanlarıdır (Bölüm 7.2).

5. Hata Avcısı

İç içe döngü hataları çoğunlukla derleyicinin göremediği **mantık** hatalarıdır: program derlenir, çalışır, ama yanlış sayıda tur döner ya da yanlış sonuç verir. Tüm dosyalar `kodlar/hata_avcisi/` altında, düzeltilmiş sürümleriyle birliktedir; hatalı dosyaların başında "BU DOSYA KASITLI HATA İÇERİR" yazar.

Tablo 7.11
Haftanın Hata Avcısı özeti (tüm çıktılar ve iletiler gerçektir)

#	Hata	Belirti (gerçek çıktı)	Neden	Çözüm
HA1	İç toplayıcı sıfırlanmıyor	satır toplamları 15, 45, 90, 150 (15, 30, 45, 60 olmalı)	Toplayıcı dış döngüden önce bir kez 0	Dış döngünün içinde bildir/sıfırla
HA2	Aynı sayaç iki döngüde	3 satır yerine 1 satır; sınırlar ters ise sonsuz döngü	İç döngü dış sayacı değiştiriyor	Her döngüye kendi sayacı (`j`)
HA3	break'in kapsamı	"İlk çözüm: $10^2 + 15^2$" ($1^2 + 18^2$ olmalı)	break yalnızca iç döngüden çıkar	Bayrak dış koşulda; ya da goto/return
HA4	Asallıkta 0, 1, 2	-3, -1, 1 "asal"; 2 "asal değil"	Sınır durumları atlanmış, sıra yanlış	n < 2, n == 2, çift: bu sırayla, döngüden önce
HA5	Seride `1 / k`	e ≈ 2.000000000000000	Tamsayı bölmesi (H4)	`1.0 / k`
HA6	`d * d <= n` taşması	INT_MAX "asal değil"; UBSan: signed integer overflow	d = 46 341'de d^2 > INT_MAX	`d <= n / d` ya da `long long`

HA1 · İç toplayıcının sıfırlanmaması ve HA2 · aynı sayaç

`H07_HA1_ic_toplayici.c` 4 × 5 tablonun (hücre i · j) satır toplamlarını ister ama `int satir_toplami = 0;` satırı dış döngüden **önce** durur. Gerçek çıktı: 15, 45, 90, 150 (satır toplamları); düzeltilmiş sürüm: 15, 30, 45, 60. İlk satır doğru çıkar (15), bu yüzden hata kolayca gözden kaçar: **ikinci satırı** test edin. Düzeltilmiş sürüm `int satir_toplami = 0;` satırını dış döngünün içine, iç döngüden hemen önceye taşır. HA2'de iç döngü de `i`'yi kullanır:

hata_avcisi/H07_HA2_ayni_sayac.c · satır 12–19

```
12      int i;
13      for (i = 1; i <= 3; i++) {
14          for (i = 1; i <= 5; i++) {  /* HATA: aynı sayaç! */
15              printf("*");
16          }
17          printf("\n");
18      }
19      printf("Döngüler bitti: i = %d\n", i);
```

Gerçek çıktı: tek satır `*****` ve "Döngüler bitti: i = 7": iç döngü i'yi 6'ya, dış güncelleme 7'ye getirir. Sınırlar ters olsaydı (`H07_HA2_ayni_sayac_sonsuz.c`: dış `i <= 5`, iç `i <= 3`) i hep 4 → 5 olur, `5 <= 5` doğru kalır: **sonsuz döngü**; `timeout 1` ile 1 saniyede 39 927 808 satır yazdı (çıkış kodu 124). Derleyici uyarmaz; çözüm: `for (int i …)` ve `for (int j …)`.

HA3 · break'in yalnızca iç döngüden çıkması

`H07_HA3_break_ic.c`, Bölüm 3.7'deki aramayı yalnız `break` ile yazar ama sonucu "ilk çözüm" diye yazdırır. Gerçek çıktı: "İlk çözüm: 10^2 + 15^2 = 325"; düzeltilmiş sürüm: "İlk çözüm: 1^2 + 18^2 = 325". Hatalı sürüm 325'in üç gösterimini de bulur; her biri `bi` ve `bj`'nin üzerine yazar ve ekranda **son** çözüm kalır. Düzeltme Bölüm 3.7'deki bayrak yöntemidir: `for (int i = 1; i <= 20 && !bulundu; i++)`.

HA4 · Asallıkta sınır durumları

`H07_HA4_asal_sinirlar.c` -3..30 arasındaki asalları, `n < 2` denetimi olmadan ve "önce çiftleri ele" kuralıyla (`if (n % 2 == 0) { asal = 0; }`, sonra tek d için `d <= n / d`) arar. Gerçek çıktı: "-3..30 arasında "asal" bulunanlar: -3 -1 1 3 5 7 11 13 17 19 23 29"; düzeltilmiş sürüm: "-3..30 arasında "asal" bulunanlar: 2 3 5 7 11 13 17 19 23 29". Hatalı sürümde n < 2 denetimi yoktur: 1 ve negatif tekler için döngü (`3 <= n / 3`) hiç çalışmaz ve bayrak 1 kalır. "Çiftler asal değildir" kuralı da 2'yi eler. Ders: sınır değerleri (0, 1, 2, negatif, tam kare: 9, 25) test kümenize **her zaman** koyun.

HA5 · Seride tamsayı bölmesi

`H07_HA5_tam_bolme.c` e'yi `e += 1 / faktoriyel;` satırıyla toplar (`faktoriyel` bir `long long`). Gerçek çıktı: "e ≈ 2.000000000000000"; düzeltilmiş sürüm: "e ≈ 2.718281828458995". `1 / faktoriyel` iki tamsayının bölümüdür: k = 0 ve 1'de 1, sonra hep 0. `e` `double` olduğu için sonucun kesirli olacağını sanmak yaygın bir yanılgıdır; dönüşüm **toplamadan sonra** olur, bölme çoktan tamsayı olarak yapılmıştır (H4). Düzeltme `1.0 / faktoriyel`.

HA6 · d * d taşması (Mühendis Gözüyle'nin gerçek örneği)

hata_avcisi/H07_HA6_karekok_tasma.c · satır 17–24

```
17      for (int d = 3; d * d <= n; d += 2) {  /* HATA: d * d taşabilir */
18          bolme++;
19          if (n % d == 0) {
20              asal = 0;
21              bolen = d;
22              break;
23          }
24      }
```

2 147 483 647 asaldır, ama hatalı program "asal = 0" dedi. d = 46 341'de 46 341 × 46 341 = 2 147 488 281 `int`'e sığmaz: **tanımsız davranış**. Bu makinede gözlenen, taşan çarpımın negatife "sarması", koşulun doğru kalması ve döngünün d = n'ye kadar sürüp n'yi "kendi böleni" bulmasıydı (yaklaşık 2,7 s); başka bir derleyicide başka bir şey olabilirdi. UBSan ilk taşmayı yakaladı: `H07_HA6_karekok_tasma.c:17:23: runtime error: signed integer overflow: 46341 * 46341 cannot be represented in type 'int'`. Düzeltilmiş sürüm (`d <= n / d`): asal = 1, bölen = 0, 23169 bölme.

6. Mühendis Gözüyle: Doğru, Ölçülmüş, Sağlam

- **Verimlilik önce algoritmadadır.** √n sınırı ve tek bölenler bölme sayısını N = 10^5'te 338 kat azalttı; `-O2` aynı naif kodu yalnızca 1,32 kat hızlandırdı. Önce yineleme sayısını azaltın, sonra ölçün; ölçmeden "hızlandırdım" demeyin.
- **Taşmayı öngörün.** Sınırlarda çarpma yerine bölme (`d <= n / d`, `kuvvet <= n / 2`), toplamadan önce denetim (`simdiki <= INT_MAX - onceki`), geniş sayaçlar (`long long`); testlerde UBSan (`-fsanitize=undefined`).

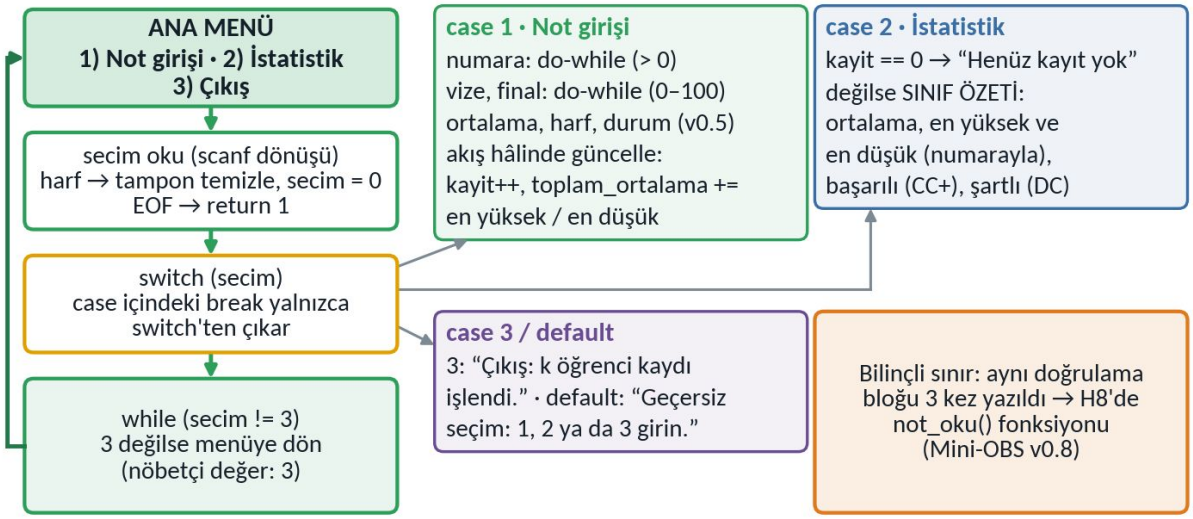
- **Güvenlik.** `DOGRU_PIN = 1234` eğitsel bir kısayoldur: gerçek sistemler PIN'i kodda ya da düz metin tutmaz, deneme sınırını sunucuda uygular. 3 denemede kilit, kaba kuvvetle tahmine karşı **arama uzayı** savunmasıdır: 10 000 olası PIN'de rastgele tahminin başarı olasılığı en çok 3/10 000.
- **Girdiye güvenmeyin.** Her `scanf` için üç sonucu (1, 0, EOF) düşünün; tek bir harf ya da dosya sonu programı kilitlememeli. `int` aralığını aşan sayıyı `scanf("%d")` ile okumak da tanımsızdır; sağlam okuma H13'te `fgets` + `strtol` ile.
- **Okunabilirlik.** Üç katlı döngüden derine inmeyin; desen formüllerini yorumla yazın; sihirli sayılar yerine adlandırılmış sabitler. Aynı bloğu üçüncü kez yazıyorsanız fonksiyon zamanı gelmiştir (H8).

7. Mini-OBS v0.7 ve Ara Proje

7.1 Mini-OBS v0.7: menü güdümlü ana döngü

v0.6 bir kez çalışıp biten bir programdı: N okunur, N öğrenci işlenir, özet yazılır. **v0.7**, aynı davranışı bir **menü döngüsüne** taşır: `1) Not girişi (bir öğrenci)`, `2) Sınıf istatistikleri`, `3) Çıkış`. Kullanıcı istediği kadar öğrenci ekleyebilir ve özeti istediği an görebilir; menü seçimi de doğrulanır. Dizi hâlâ yok (H10): istatistikler v0.6'daki gibi akış hâlinde tutulur (Şekil 7.7). Kaynak: `91_Proje_MiniOBS/v0.7/` (kopyası `kodlar/proje/`). Kurallar (RTEÜ tablosu; Recep Tayyip Erdoğan Üniversitesi, 2025; %40 vize + %60 final, final barajı 50, `(int)(ortalama + 0.5)` yuvarlama, şartlı DC) v0.5–v0.6 ile **aynıdır**.

Şekil 7.7
Mini-OBS v0.7: menü döngüsü, case'ler ve akış hâlinde istatistik



Kabul ölçütleri. Sıfır uyarı; menüde sayı olmayan ya da 1–3 dışı girdi "Geçersiz seçim" iletisiyle menüye döner; kayıt yokken istatistik istenirse "Henüz kayıt yok" yazılır (sıfıra bölme yok); not girişindeki her alan v0.6'daki gibi doğrulanır; çıkışta işlenen kayıt sayısı yazılır; girdi yarıda biterse (EOF) "Girdi sona erdi." ve çıkış kodu 1. Testlerin gerçek çıktıları `mini_obs_v0_7_ciktilar.txt` dosyasındadır:

Tablo 7.12
Mini-OBS v0.7 test senaryoları (girdi dosyasıyla gerçek çalıştırma)

Test	Senaryo	Kayıt	Sınıf ort.	Başarılı	Şartlı	Çıkış
T1	normal akış: dört öğrenci eklenir, istatistik istenir, çıkılır	4	68.20	2 / 4	1 / 4	0
T2	geçersiz menü girdileri: harf, aralık dışı, ondalıklı; sonra çıkış	0	—	—	—	0

Test	Senaryo	Kayıt	Sınıf ort.	Başarılı	Şartlı	Çıkış
T3	kayıt yokken istatistik; geçersiz not girdileriyle bir kayıt; yeniden istatistik	1	59.94	0 / 1	0 / 1	0
T4	sınır değerler (v0.6 T4 ile aynı dört öğrenci): final tam 50, 59.5 → 60, 49.6 → 50, 89.5 → 90	4	64.65	3 / 4	1 / 4	0
T5	girdi dosyası not girişinin ortasında biter (EOF)	—	—	—	—	1

Bilinçli sınırlar. (1) Numara, vize ve final için neredeyse aynı doğrulama bloğu üç kez yazıldı (menü seçimiyle dört): H8'de `not_oku()` gibi bir fonksiyona dönüşecek (v0.8). (2) Menüye `2.5` yazılırsa `scanf("%d")` 2'yi okur, `.5` tamponda kalır ve sonraki turda geçersiz seçim sayılır (T2'de görülür); sağlam satır okuma H13'te. (3) Kayıtlar saklanmaz; aynı numara iki kez girilebilir, kayıt silinemez (H10–H14).

7.2 Ara proje (eşli): yönergenin özeti

MİNİ-OBS PROJESİ · **Ara proje: menü güdümlü bir program**

Ara proje, kontrol akışının (H5–H7) bütünleştirilmesidir ve eşli yapılır. İki seçenek vardır: **A) Menü güdümlü ATM simülasyonu** (bakiye sorgulama, para yatırma/çekme, 3 hatalı PIN'de kilit, tüm girdilerin doğrulanması) ya da **B) Mini-OBS v0.7** (Bölüm 7.1). Ayrıntılı yönerge, kabul ölçütleri, test tablosu ve 100 puanlık rubrik `92_Degerlendirme/AP1_Ara_Proje_Yonergesi.md` dosyasındadır; teslim tarihi ve biçimi derste duyurulur.

- **Teslim:** kaynak kod (tek .c dosyası, sıfır uyarı), test senaryoları ve gerçek çıktıları (en az 6 senaryo: normal, sınır, hatalı girdi, EOF), 1–2 sayfalık kısa rapor (tasarım, akış diyagramı, karşılaşılan hatalar, iş bölümü).
- **Puanlama (100):** işlevsel doğruluk 35 · girdi doğrulama ve sağlamlık 20 · kod kalitesi 15 · testler 15 · rapor 10 · ek özellik 5.
- **Akademik dürüstlük:** iki kişi birlikte yazar, ikisi de kodun her satırını açıklayabilmelidir; başka gruplardan ya da yapay zekâ araçlarından alınan her parça raporda belirtilir. Örnek 14 bir referanstır: kopyası değil, kendi tasarımınız değerlendirilir.

8. Alıştırmalar ve Atölye

Kodlu alıştırmalar `kodlar/alistirmalar/` altındadır; yıldızlar zorluğu gösterir.

Alıştırma 7.1 ★☆☆ *Çıktı tahmini*

Programın çıktısını çalıştırmadan tahmin edin (`H07_A01_cikti_tahmini_x.c`). 4 × 4 ızgarayı kâğıtta çizip X olacak hücreleri işaretleyin.

alistirmalar/H07_A01_cikti_tahmini_x.c · satır 11–20

```
    for (int i = 1; i <= 4; i++) {
        for (int j = 1; j <= 4; j++) {
            if (i == j || i + j == 5) {
                printf("X");
            } else {
                printf(".");
            }
        }
        printf("\n");
    }
```

Alıştırma 7.2 ★★☆ *Çıktı tahmini*

`toplam` son olarak kaçtır (`H07_A02_break_continue_ic_ice.c`)? Her (i, j) için "eklendi / atlandı / iç döngü bitti" izleme tablosu kurun. `break` ve `continue` hangi döngüyü etkiliyor?

alistirmalar/H07_A02_break_continue_ic_ice.c · satır 12–24

```c
    int toplam = 0;
    for (int i = 1; i <= 4; i++) {
        for (int j = 1; j <= 4; j++) {
            if (j > i) {
                break;
            }
            if ((i + j) % 2 == 1) {
                continue;
            }
            toplam += i * j;
        }
    }
    printf("toplam = %d\n", toplam);
```

Alıştırma 7.3 ★★☆ *Hata bul*

n = 3 için ters piramit (5, 3, 1 yıldız) isteniyor; program başka bir desen çiziyor (`H07_A03_hata_bul_ters_piramit.c`). Satır–boşluk–yıldız tablosu kurup iki formül hatasını bulun.

alistirmalar/H07_A03_hata_bul_ters_piramit.c · satır 16–24

```c
    for (int i = 1; i <= n; i++) {
        for (int b = 1; b <= n - i; b++) {
            printf(" ");
        }
        for (int y = 1; y <= 2 * i + 1; y++) {
            printf("*");
        }
        printf("\n");
    }
```

Alıştırma 7.4 ★★☆ *Hata bul*

2–20 arasındaki asallar beklenirken program yalnızca "2 3" yazıyor (`H07_A04_hata_bul_asal_listesi.c`). Neden? Hangi satır nereye taşınmalı?

alistirmalar/H07_A04_hata_bul_asal_listesi.c · satır 12–23

```c
    int asal = 1;
    for (int n = 2; n <= 20; n++) {
        for (int d = 2; d < n; d++) {
            if (n % d == 0) {
                asal = 0;
                break;
            }
        }
        if (asal) {
            printf("%d ", n);
        }
    }
```

Alıştırma 7.5 ★★☆ *Parsons*

Satırları sıralayıp girintileyerek “1 / 2 2 / 3 3 3 / 4 4 4 4” çıktısını üreten programı kurun; bir satır fazladır (`H07_A05_parsons_sayi_ucgeni.c`).

alistirmalar/H07_A05_parsons_sayi_ucgeni.c · satır 14–19

```
14          printf("\n");
15      for (int i = 1; i <= 4; i++) {
16              printf("%d ", j);
17          for (int j = 1; j <= i; j++) {
18      }
19              printf("%d ", i);
```

Alıştırma 7.6 ★★☆ *Kod tamamla*

Boşlukları doldurun: 5 × 5 çarpım tablosunun yalnızca köşegeni ve üstü (j ≥ i), diğer hücreler 4 boşluk (`H07_A06_kod_tamamla_ust_ucgen.c`). İç gövde kaç kez çalışır, kaç kez sayı yazar?

alistirmalar/H07_A06_kod_tamamla_ust_ucgen.c · satır 12–21

```
12      for (int i = 1; i <= 5; i++) {
13          for (int j = 1; j <= ____; j++) {
14              if (____) {
15                  printf("%4d", ____);
16              } else {
17                  printf("____");
18              }
19          }
20          ____;
21      }
```

Alıştırma 7.7 ★☆☆ *Kod yaz*

n × n **satranç tahtası** deseni yazın: (i + j) çiftse `#`, tekse `.`. n = 6 için ilk satır `#.#.#.`, ikinci satır `.#.#.#`.

Alıştırma 7.8 ★★☆ *Kod yaz*

N’ye kadar **ikiz asalları** (p ve p + 2 ikisi de asal) listeleyin ve sayın. N = 100 için 8 çift: (3, 5) … (71, 73). Asallık testini kaç kez yazdınız? H8’de bunu tek fonksiyona indireceğiz.

Alıştırma 7.9 ★★☆ *Kod yaz*

n ≥ 2’yi **asal çarpanlarına** ayırın: 360 = 2 x 2 x 2 x 3 x 3 x 5; 97 = 97. İpucu: dış döngü bölen adayını, iç `while` aynı bölenin kaç kez böldüğünü dolaşır; `d <= n / d` bitince kalan n > 1 ise asaldır.

Alıştırma 7.10 ★★★ *Kod yaz*

Goldbach doğrulaması. 4’ten 10 000’e kadar her çift sayının iki asalın toplamı olarak yazılabildiğini doğrulayın; 20’ye kadar olanların en küçük ayrışımını yazdırın (8 = 3 + 5). Hangi n için en küçük p en büyüktür? (Goldbach sanısı kanıtlanmamıştır; doğrulamanız bir kanıt değil, bir sınamadır.)

Alıştırma 7.11 ★★★ *Kod yaz*

Heron (Babil) yöntemiyle karekök. a > 0 için x ← (x + a/x) / 2 yinelemesini ardışık tahminler arasındaki fark 10^{-12} altına inene dek sürdürün; her yinelemeyi yazın ve `sqrt(a)` ile karşılaştırın. a = 2 için kaç yineleme? Hangi döngü türü doğal?

Alıştırma 7.12 ★★★ *Kod yaz*

Menü güdümlü desen çizici. `1) Dikdörtgen 2) Dik üçgen 3) Piramit 0) Çıkış`; seçimden sonra n (1–20) okunur ve doğrulanır. Geçersiz seçim ve n menüye döndürür; EOF'ta "Girdi sona erdi." ile çıkılır. Ara proje için iyi bir ısınmadır.

ALGORİTMA ATÖLYESİ · **Atölye etkinlikleri**

- **Desen üretici yarışması (20 dk, eşli).** Öğretim üyesi tahtaya bir desen çizer (kum saati, içi boş elmas, sayı piramidi 1 / 121 / 12321). Eşler önce satır tablosunu, sonra kodu yazar; ölçüt **en okunur kod**: doğru çıktı, açık formül yorumları, en az değişken.
- **Ölçüm deneyi (25 dk).** `H07_K06_olcum_deneyi.c`'yi kendi bilgisayarınızda -O0 ve -O2 ile derleyin; naif yöntemi N = 10^3, 10^4, 10^5 için, iyileştirilmiş yöntemi 10^6 ve 10^7 için 5 tekrarla çalıştırın. Medyanları tabloya ve logaritmik bir grafiğe dökün, Bölüm 3.9 ile karşılaştırın. 10^6 ölçeğinde tek bir sayıyı (999 983, bir asal) iki yöntemle 1000 kez test edip tek test süresini hesaplayın: neden tekrar gerekir?
- **Ara proje başlangıcı (30 dk, eşli).** Seçeneği belirleyin, akış diyagramını çizin, test tablosunun ilk altı satırını yazın; kodlamaya diyagram onaylandıktan sonra geçin.

9. Kendini Değerlendirme

Yapabiliyorum…	
☐ İç içe bir döngünün çalışma sırasını ızgara modeliyle anlatabiliyorum.	☐ Dikdörtgen ve üçgen döngülerin yineleme sayısını (n·m, n(n+1)/2) hesaplayıp sayaçla doğrulayabiliyorum.
☐ Bir deseni satır–boşluk–yıldız tablosuna ve formüle dönüştürüp kodlayabiliyorum.	☐ Naif ve √n asallık testlerini sınır durumlarıyla (0, 1, 2) doğru yazabiliyorum.
☐ ε eşiğiyle biten seri döngüsü yazıp iç içe döngüden bayrakla (ya da goto ile) çıkabiliyorum.	☐ clock() ile ölçüp sonucu raporlayabiliyor, doğrulamalı menü programı yazabiliyorum.

10. AP-2 Köprüsü ve Merak Kutusu

AP-2 KÖPRÜSÜ · **İç içe döngüden algoritma çözümlemesine**

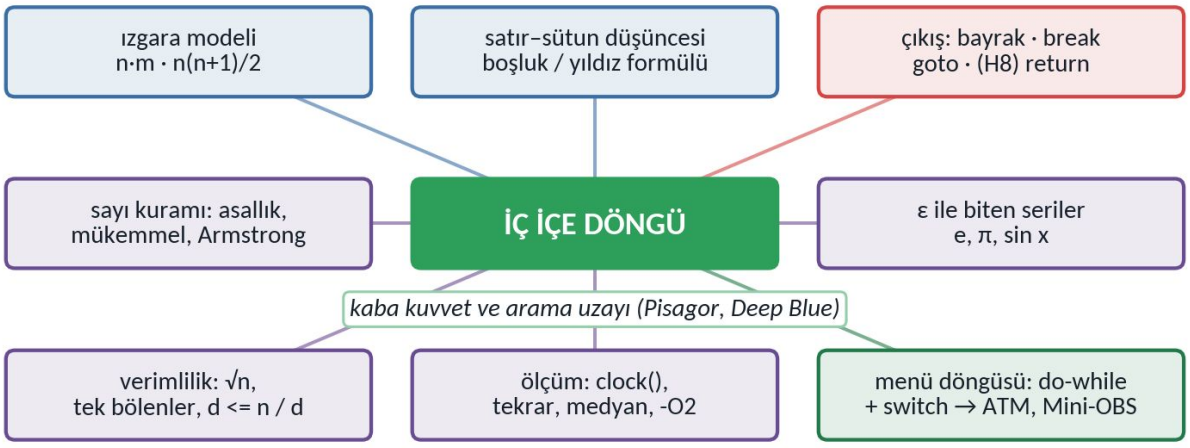
- İki katlı iç içe döngünün N^2 ve N(N+1)/2 yinelemesi, AP-2'de **kabarcık** ve **seçme sıralamasının** karesel karmaşıklığının sezgisel karşılığıdır: seçme sıralaması, H6'daki "en küçüğü bul" döngüsünü bir dış döngünün içine koymaktan ibarettir.
- √n asallık testi, AP-2'de **Eratosthenes Kalburu** ile karşılaştırılacak: tek tek test etmek yerine bir aralıktaki tüm bileşikleri dizide işaretleyerek eleyen yöntem (H10'daki dizilerden sonra).
- Kaba kuvvet ve arama uzayı (Pisagor, Deep Blue), AP-2'de **geri izleme** (backtracking) ve **budama** tartışmalarının zeminidir: Örnek 13'teki `break`'ler, arama uzayını budamanın ilk örnekleridir.

> MERAK KUTUSU · **Kaç asal var? Ölçümümüzden bir bulmaca**
>
> Ölçüm deneyimiz yan ürün olarak 0..N arasındaki asal sayısını, π(N)'yi verdi (Tablo 7.3): $\pi(10^3) = 168$, $\pi(10^4) = 1229$, $\pi(10^5) = 9592$, $\pi(10^6) = 78\,498$, $\pi(10^7) = 664\,579$. Asallar seyrekleşir ama tükenmez (Öklid'in kanıtı). **Asal sayı teoremine** göre π(N), büyük N için $N / \ln N$'ye yaklaşır: $10^7 / \ln 10^7 \approx 620\,421$, gerçek değerin yaklaşık %93'ü; oran N büyüdükçe yavaşça 1'e yaklaşır.

11. Özet, Kavram Haritası ve Terim Sözlüğü

Özet. İç içe döngüde iç döngü, dış döngünün her turunda baştan sona çalışır; ızgarada her hücre bir yinelemedir (n·m, n(n+1)/2). Desenler satır tablosu ve formülle kurulur. Asallıkta önce sınırlar (0, 1, 2), sonra √n'e kadar tek bölenler; `d <= n / d` taşmaz. Seriler ε ile biter. `break` yalnızca en içteki döngüden çıkar. Menü = `do-while` + `switch` + doğrulama. Hız iddiası ölçülür: yineleme sayısı değişmez, süre değişir.

Şekil 7.8
Haftanın kavram haritası



Terim sözlüğü

Tablo 7.13
Haftanın terimleri (yol haritası Bölüm 6.8 ile uyumlu)

Türkçe	İngilizce	Kısa açıklama
iç içe döngü	nested loop	Gövdesinde başka bir döngü bulunan döngü
yineleme (tur)	iteration	Döngü gövdesinin bir kez çalışması
asal sayı; bölen	prime number; divisor	Yalnızca 1'e ve kendisine bölünen n > 1; n'yi kalansız bölen
kaba kuvvet; arama uzayı	brute force; search space	Tüm adayları denemek; adayların kümesi
ε (epsilon) eşiği	tolerance (epsilon)	Döngüyü durduran küçük sınır değer
menü güdümlü program	menu-driven program	Seçime göre iş yapan, çıkışa kadar dönen program
işlemci zamanı; duvar saati	CPU time; wall-clock time	clock() ile ölçülen; gerçek geçen süre
ölü kod eleme	dead code elimination	Derleyicinin sonucu kullanılmayan kodu silmesi
tanımsız davranış; taşma	undefined behavior; overflow	Standardın sonucunu belirlemediği durum; tipe sığmama

Büyük Sorunun yanıtı. Bir sayının asal olup olmadığını en hızlı nasıl anlarız? Gereksiz bölmeleri hiç yapmayarak: bölen çiftlerinden biri √n'den büyük olamayacağı için √n'de durmak, 2'yi ayrı ele alıp yalnızca tek bölenleri denemek. 2 147 483 647 için naif test 2 147 483 645 bölme yaptı; iyileştirilmiş test 23 169 bölmeyle bitirdi (gerçek çıktılar). **"Daha hızlı" iddiamızı nasıl ölçeriz?** İki ölçüyle: makineden bağımsız **yineleme sayısıyla** (sayaçla) ve makineye bağlı **süreyle** (`clock()`, tekrar, medyan, ortam ve seçeneklerin raporlanması). Deneyimizde N = 10^5'te bölme sayısı 338 kat, süre yaklaşık 192 kat azaldı.

> NOT · **H8'e köprü: aynı kodu kaç kez yazdık?**
>
> Bu hafta `n % d == 0` biçimindeki bölen denetimini (asallık testi, bölen toplamı, bölen çiftleri) merdiven, kavram, Hata Avcısı ve alıştırma dosyalarından 12 tanesinde, toplam 17 kez yeniden yazdık (sayım: `kodlar/` altındaki dosyalar). ATM'de `scanf` dönüşünü denetleyip tamponu temizleyen blok dört, Mini-OBS'te dört kez tekrarlandı. Bir hatayı (ör. HA4'teki sınır durumu) düzeltmek için hepsini tek tek bulmak zorundayız. H8'in yanıtı **fonksiyonlardır**: `is_prime(n)` bir kez yazılır, her yerde çağrılır, bir kez test edilir; Örnek 14 ve Mini-OBS fonksiyonlarla yeniden yapılandırılacak.

Gelecek hafta: Fonksiyonlar I — asallık testi ve girdi doğrulama fonksiyonlara taşınacak; ATM ve Mini-OBS yeniden yapılandırılacak (v0.8). Hazırlık: Örnek 6, 11, 14'ü yeniden okuyun, Alıştırma 7.8'i tamamlayın.

12. Kaynaklar

Böhm, C., & Jacopini, G. (1966). Flow diagrams, Turing machines and languages with only two formation rules. *Communications of the ACM*, *9*(5), 366–371. https://doi.org/10.1145/355592.365646

Campbell, M., Hoane, A. J., Jr., & Hsu, F.-h. (2002). Deep Blue. *Artificial Intelligence*, *134*(1–2), 57–83. https://doi.org/10.1016/S0004-3702(01)00129-1

Deitel, P., & Deitel, H. (2022). *C how to program: With case studies introducing applications and systems programming* (9th ed., global ed.). Pearson.

Dijkstra, E. W. (1968). Go to statement considered harmful. *Communications of the ACM*, *11*(3), 147–148. https://doi.org/10.1145/362929.362947

IBM. (t.y.). *Deep Blue*. IBM History. https://www.ibm.com/history/deep-blue

ISO/IEC. (2018). *Information technology — Programming languages — C* (ISO/IEC Standard No. 9899:2018). International Organization for Standardization.

Knuth, D. E. (1974). Structured programming with go to statements. *ACM Computing Surveys*, *6*(4), 261–301. https://doi.org/10.1145/356635.356640

Microsoft. (t.y.). *clock*. Microsoft Learn: C runtime library reference. https://learn.microsoft.com/en-us/cpp/c-runtime-library/reference/clock

Recep Tayyip Erdoğan Üniversitesi. (2025). *Recep Tayyip Erdoğan Üniversitesi ön lisans ve lisans eğitim-öğretim ve sınav yönetmeliği* (Resmî Gazete, 14 Aralık 2025, Sayı: 33107; son değişiklik: Resmî Gazete, 27 Şubat 2026, Sayı: 33181).

Shannon, C. E. (1950). Programming a computer for playing chess. *The London, Edinburgh, and Dublin Philosophical Magazine and Journal of Science*, *41*(314), 256–275. https://doi.org/10.1080/14786445008521796

The Linux Kernel Documentation. (t.y.). *Linux kernel coding style*. https://www.kernel.org/doc/html/latest/process/coding-style.html